

Smarter Page Turner: Ein audiobasiertes Echtzeitsystem zur automatisierten Partiturverfolgung von Violinmusik

Studienarbeit

des Studiengangs **Elektrotechnik - Automation**

an der Dualen Hochschule Baden-Württemberg Mannheim

von

Samuel Geffert

07. April 2026

Bearbeitungszeitraum:	12 Wochen
Matrikelnummer, Kurs:	4203964, TEL23AT1
Dualer Partner:	KHS GmbH, Worms
Betreuer:	Prof Dr. Rüdiger Heintz

Erklärung

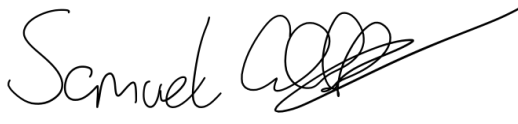
Ich versichere hiermit, dass ich meine Studienarbeit mit dem Thema:

*Smarter Page Turner: Ein audiobasiertes Echtzeitsystem zur automatisierten
Partiturerfolgung von Violinmusik*

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Dabei wurde unter anderem Generative AI (Google Gemini) als Hilfsmittel zur sprachlichen Überarbeitung und zur Einholung von Formulierungsvorschlägen genutzt. Zudem erfolgte die Ausarbeitung des Programms mit Unterstützung von Claude Code. Die inhaltliche Erarbeitung sowie die finale Auswahl und Prüfung der Texte und des Codes erfolgten vollständig eigenständig.

Mannheim, den 07. April 2026

A handwritten signature in black ink, appearing to read 'Samuel Geffert', with a long horizontal stroke extending to the right. Below the signature is a thin horizontal line.

Samuel Geffert

Zusammenfassung

Diese Studienarbeit befasst sich mit der Konzeption, Implementierung und Evaluation eines vollautomatisierten Echtzeit-Systems zum Umblättern von digitalen Notenblättern (Page Turner) auf Basis von akustischem Score Following. Das System ist dabei primär für den Einsatz mit der Violine konzipiert, da die permanente Bogenführung und das Greifen der Saiten eine freihändige Lösung hier besonders erforderlich machen. Ziel ist es, den musikalischen Fluss während des Übeprozesses oder bei Aufführungen nicht durch manuelles Blättern oder die zusätzliche motorische Koordination von Fußpedalen zu stören. Hierzu synchronisiert das System das fortlaufende Mikrofonsignal des Musikers kontinuierlich mit einer digitalen Referenzpartitur.

Um die hohen Anforderungen an Latenz und Ressourceneffizienz zu erfüllen, gliedert sich die Systemarchitektur strikt in zwei Bereiche: Eine vorbereitende Offline-Pipeline und eine asynchrone Echtzeit-Laufzeitumgebung. Die Offline-Pipeline überführt dabei PDF- oder MusicXML-Partituren mittels Optical Music Recognition (OMR) und Audiosynthese in L_2 -normierte, spektrale Chroma-Merkmale. Zur Laufzeit bildet ein inkrementeller Online Dynamic Time Warping (ODTW) Algorithmus den Kern des Trackings. Dieser ermöglicht eine robuste Positionsbestimmung, welche sowohl temporale Abweichungen als auch Dynamikschwankungen in der Spielweise zuverlässig kompensiert.

Ein besonderer Fokus der Arbeit liegt zudem auf der Praxistauglichkeit: Ein JIT-kompilierter Subsequence-DTW-Scan (sDTW) fungiert als intelligenter Recovery-Mechanismus. Dieser stellt bei non-linearen Sprüngen im Notentext oder signifikanten Spielfehlern eine schnelle und latenzfreie Re-Synchronisation sicher. Die abschließende Evaluation belegt die Echtzeitfähigkeit des Systems sowie eine hohe Robustheit gegenüber Hintergrundrauschen und Tempovariationen. Limitationen des gewählten Ansatzes zeigen sich primär bei hochgradig repetitiven Musikstücken, da die aggregierenden Chroma-Merkmale hier zu strukturellen Mehrdeutigkeiten bei der globalen Positionsfindung führen können.

Abstract

This study focuses on the design, implementation, and evaluation of a fully automated, real-time page-turning system for digital sheet music based on acoustic score following. The system is primarily designed for use with the violin, as the continuous bowing and fingering required for string instruments make a hands-free solution particularly essential. The primary objective is to prevent disruptions to the musical flow during practice sessions or performances caused by manual page turning or the physical coordination required for foot pedals. To achieve this, the system continuously synchronizes the musician's live microphone signal with a digital reference score.

To meet strict latency and resource efficiency requirements, the system architecture is strictly divided into two components: a preparatory offline pipeline and an asynchronous real-time execution environment. The offline pipeline converts PDF or MusicXML scores into L_2 -normalized spectral chroma features using Optical Music Recognition (OMR) and audio synthesis. During runtime, an incremental Online Dynamic Time Warping (ODTW) algorithm forms the core of the tracking process. This enables robust position tracking that reliably compensates for temporal deviations and dynamic fluctuations in the performance.

Furthermore, a special focus of this work lies on practical applicability: a JIT-compiled Subsequence DTW (sDTW) scan acts as an intelligent recovery mechanism. This ensures rapid and latency-free re-synchronization in the event of non-linear jumps in the score or significant playing errors. The final evaluation demonstrates the system's real-time capabilities and its high robustness against background noise and tempo variations. Limitations of this approach primarily arise in highly repetitive musical pieces, as the aggregating nature of chroma features can lead to structural ambiguities during global position recovery.

Inhaltsverzeichnis

Formelverzeichnis	VII
Abkürzungsverzeichnis	IX
Abbildungsverzeichnis	XI
Tabellenverzeichnis	XII
Listings	XIII
1 Einleitung	1
1.1 Zielsetzung	1
1.2 Aufbau der Arbeit	2
2 Theoretische Grundlagen und Stand der Technik	3
2.1 Optical Music Recognition und Partitur-Formate	3
2.2 Audiosignalverarbeitung und Merkmalsextraktion	5
2.3 Score Following	9
2.4 Dynamic Time Warping (DTW)	11
3 Methodik und Systemarchitektur	17
3.1 Anforderungsanalyse	17
3.2 Gesamtkonzept und methodische Entscheidungen	18
3.3 Die Score-Pipeline (Offline-Vorverarbeitung)	20
3.4 Architektur des Live-Systems (Echtzeit-Komponente)	23
4 Algorithmus-Design: Echtzeit-Tracking und Recovery	27
4.1 Merkmalsabgleich und Vektordistanz	27
4.2 Inkrementelle Pfadfindung via ODTW	28
4.3 Der Recovery-Mechanismus (Re-Synchronisation)	30
5 Evaluation und Messergebnisse	33
5.1 Versuchsaufbau und Methodik	33
5.2 Robustheitsanalyse des Tracking-Algorithmus	34
5.3 Analyse der Echtzeitfähigkeit und Systemlatenz	36

5.4	Validierung der funktionalen Anforderungen	37
6	Fazit und Ausblick	39
6.1	Zusammenfassung	39
6.2	Zukünftige Optimierungspotenziale	40
	Literatur	i

Formelverzeichnis

$C(i, j)$	Lokale Kostenmatrix (Vektordistanz an Position i, j).
d	Dämpfungsfaktor (Damping).
$d_{\cos}(u, v)$	Kosinus-Distanzfunktion.
Δf	Frequenzauflösung (Bandbreite eines Bins).
Δt	Zeitliche Update-Rate des Trackers.
$d_{\text{euc}}(u, v)$	Euklidische Distanzfunktion.
$D(i, j)$	Kumulierte Kostenmatrix im DTW-Algorithmus.
f_s	Abtastrate des Audiosignals.
i	Index des Live-Signals.
j	Index der Referenzsequenz.
M	Länge der Referenzsequenz.
N	Länge der Suchsequenz.
n	Abtastwert (Sample-Index) im Zeitbereich.
N_{FFT}	Fenstergröße (Anzahl der Samples) bei der Frequenzanalyse.
N_{hop}	Hop-Size (Blockgröße).

t_{win}	Absolute zeitliche Fensterdauer.
u, v	Beliebige Merkmalsvektoren.
$ v _2$	Euklidische Norm (L_2 -Norm) eines Vektors.
$w[n]$	Fensterfunktion.
w_{skip}	Penalty-Gewicht für das Überspringen.
w_{step}	Penalty-Gewicht für diagonalen Schritt.
w_{wait}	Penalty-Gewicht für Stillstand.
X	Referenzsequenz (Partitur).
$X(m, \omega)$	Spektrogramm-Matrix (STFT).
$x[n]$	Zeitdiskretes Audiosignal.
x, y	Live-Vektor (x) und Referenz-Vektor (y).
Y	Suchsequenz / Live-Audiosignal.

Abkürzungsverzeichnis

BPM	Beats per Minute.
CPU	Central Processing Unit.
CQT	Constant-Q Transform.
DFT	Diskrete Fourier-Transformation.
DTW	Dynamic Time Warping.
FFT	Fast Fourier Transform.
GUI	Graphical User Interface.
HMM	Hidden Markov Model.
IMSLP	International Music Score Library Project.
MFCC	Mel-Frequency Cepstral Coefficient.
MIDI	Musical Instrument Digital Interface.
MusicXML	Auf der Extensible Markup Language (XML) basierendes Dateiformat.
OCR	Optical Character Recognition.
ODTW	Online Dynamic Time Warping.
OMR	Optical Music Recognition.

PDF	Portable Document Format.
sDTW	Subsequence-Dynamic Time Warping.
STFT	Short-Time Fourier Transform.
XML	Extensible Markup Language.

Abbildungsverzeichnis

2.1	Schematische Darstellung des OMR-Prozesses	4
2.2	Zuordnung der Frequenz-Bins zu den 12 Pitch-Classes eines Chroma-Vektors [19]	7
2.3	Geometrischer Vergleich zwischen Euklidischer Distanz und Kosinus-Distanz bei zweidimensionalen, L_2 -normierten Vektoren.	8
2.4	Zeitliche Entwicklung der Algorithmen (DTW, HMM, KI) in Abhängigkeit vom verwendeten Input-Feature.	11
2.5	Visualisierungsbeispiel des Dynamic Time Warping-Algorithmus.	12
2.6	Visualisierungsbeispiel des Subsequence Dynamic Time Warping-Algorithmus.	13
2.7	Visualisierungsbeispiel eines ODTW-Schritts.	15
3.1	Zweigeteilte Systemarchitektur: Datenfluss von der Offline-Score-Pipeline zur Echtzeit-Tracking-Komponente.	19
3.2	Ausschnitt einer berechneten Chroma-Matrix aus dem synthetisierten Referenz-Audio.	21
3.3	Vergleich von Chroma-Matrizen in musikalischen Pausen: Chroma-Signatur der durch das Rauschmodell korrigierten Referenz-Synthese (unten) und eine reale Mikrofonaufnahme (oben).	22
5.1	Verlauf der Tracking-Pfade bei variierendem Rauschpegel σ und konstantem Tempo.	34
5.2	Verlauf der Tracking-Pfade bei variierenden Tempofaktoren und konstantem Hintergrundrauschen ($\sigma = 0.05$).	35

Tabellenverzeichnis

3.1	Struktur und Datentypen des .npz-Referenzarchivs, unterteilt in funktionale Kernkomponenten und UI-Metadaten.	23
5.1	Zusammenfassung der Testergebnisse für sieben simulierte Positionssprünge zur Auslösung des Recovery-Mechanismus.	36
6.1	Übersicht der identifizierten Optimierungspotenziale für zukünftige Systemiterationen.	40

Listings

4.1	Kernschleife des inkrementellen ODTW-Trackers zur Berechnung der akkumulierten Pfadkosten pro Live-Frame.	28
4.2	Numba-JIT-kompilierte Kernschleife des Subsequence-DTW-Scans zur globalen Re-Synchronisation.	30

1 Einleitung

Das manuelle Umblättern von Notenblättern ist für Musiker ein bekanntes Problem, das besonders beim regelmäßigen Üben störend wirkt. Da bei den meisten Instrumenten beide Hände durchgehend benötigt werden, zwingt das Umblättern den Musiker dazu, kurzzeitig mit dem Spielen aufzuhören. Diese Unterbrechung stört den musikalischen Fluss und lenkt ab [1]. Besonders bei schnellen oder komplexen Passagen lässt sich das Notenblatt oft gar nicht per Hand umblättern, ohne hörbare Pausen zu verursachen [2].

Digitale Noten auf Tablets reduzieren zwar den Papieraufwand und senken Studien zufolge die kognitive Belastung im Vergleich zu analogen Noten, lösen das eigentliche Bedienproblem aber nicht vollständig [3]. Häufig eingesetzte Bluetooth-Fußpedale erfordern eine zusätzliche motorische Koordination. Dies wird von vielen Nutzern als wenig intuitiv empfunden und verlangt eine gewisse Eingewöhnungszeit. Zudem stört die zusätzliche Aufgabe des Pedaldrucks die Konzentration auf das eigentliche Musizieren [3].

Um dieses Problem zu lösen, bietet sich das sogenannte *Score Following* (Partiturverfolgung) an. Neben dem reinen Umblättern wird diese Technologie in der Musikinformatik auch genutzt, um beispielsweise Solisten automatisch zu begleiten [4], digitale Lernsysteme für Instrumente umzusetzen [5] oder interaktive Konzertführer zu entwickeln [6]. Für den Einsatz als praktischer Page Turner muss ein solches System jedoch strenge Anforderungen erfüllen: Es muss in Echtzeit arbeiten, unempfindlich gegenüber Hintergrundrauschen sein und auch dann zuverlässig funktionieren, wenn das Tempo beim Spielen variiert.

1.1 Zielsetzung

Das primäre Ziel dieser Studienarbeit ist die Entwicklung und Implementierung eines robusten, vollautomatisierten Echtzeit-Page-Turners, der in erster Linie für den Einsatz mit der Violine konzipiert ist. Da das Greifen der Saiten und die kontinuierliche Bogenführung bei Streichinstrumenten beide Hände permanent binden, ist der Bedarf

nach einer freihändigen Lösung hier besonders hoch. Obwohl der Systemfokus auf der Violine liegt, soll die Architektur so generisch gehalten werden, dass die Anwendung für andere Instrumente als zweitrangiges Ziel grundsätzlich möglich bleibt.

Das System soll in der Lage sein, die aktuelle Position des Musikers innerhalb einer Partitur ausschließlich auf Basis des akustischen Mikrofonsignals zu verfolgen und bei Erreichen des Seitenendes selbstständig ein Steuersignal zum Umblättern zu generieren. Die algorithmische Grundlage für die Echtzeit-Synchronisation bildet dabei das Online Dynamic Time Warping (ODTW).

Ein besonderer Fokus der Konzeption liegt auf der Praxistauglichkeit während des alltäglichen Übeprozesses. Da Musiker ein Stück selten ausschließlich linear von Anfang bis Ende durchspielen, sondern gezielt einzelne Phrasen wiederholen oder zu vorherigen Takten zurückspringen, muss das System solche non-linearen Abläufe abbilden können. Einen zentralen technischen Schwerpunkt bildet daher die Entwicklung eines Recovery-Mechanismus. Dieser soll gewährleisten, dass das System manuelle Positionswechsel des Musikers erkennt und sich nach einem Sprung schnellstmöglich wieder mit der tatsächlichen Spielposition synchronisiert.

Um die Praxistauglichkeit quantifizierbar zu belegen, wird abschließend die Ausfallsicherheit des Gesamtsystems evaluiert. Im Vordergrund der Untersuchung stehen dabei die Stabilität des Trackers bei Tempovariationen und Hintergrundrauschen sowie die Zuverlässigkeit und Reaktionszeit der Re-Synchronisation bei Sprüngen in der Partitur.

1.2 Aufbau der Arbeit

Die vorliegende Arbeit ist in sechs Kapitel gegliedert. Nach dieser Einleitung erläutert Kapitel 2 zunächst die theoretischen Grundlagen der optischen Musikererkennung, der Audiosignalverarbeitung sowie der Merkmalsextraktion. Zudem wird der aktuelle Stand der Technik im Bereich Score Following vorgestellt und das Dynamic Time Warping als zentrales Verfahren eingeführt. Kapitel 3 beschreibt im Anschluss die Gesamtarchitektur des entwickelten Systems, aufgeteilt in die Vorverarbeitung und die Echtzeit-Anwendung. In Kapitel 4 wird der Kern des Systems, der inkrementelle ODTW-Algorithmus samt Recovery-Mechanismus, detailliert erklärt. Die anschließende Evaluation der Leistungsfähigkeit und Ausfallsicherheit erfolgt in Kapitel 5. Kapitel 6 fasst die Ergebnisse abschließend zusammen und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2 Theoretische Grundlagen und Stand der Technik

Das folgende Kapitel beschreibt die theoretischen Voraussetzungen, die für die Konzeption und Implementierung eines audiobasierten Systems zur Partiturverfolgung notwendig sind. Zunächst werden die Verfahren der optischen Musikerkennung (engl. Optical Music Recognition (OMR)) sowie die gängigen digitalen Austauschformate für Notenmaterial erläutert. Darauf folgt eine detaillierte Darstellung der Methoden zur digitalen Audiosignalverarbeitung, wobei der Fokus auf der Extraktion robuster Merkmale für den Zeit-Frequenz-Abgleich liegt. Abschließend wird der aktuelle Forschungsstand im Bereich des Score Following dargelegt, um die methodischen Ansätze für den Echtzeit-Abgleich im Kontext bestehender Paradigmen einzuordnen.

2.1 Optical Music Recognition und Partitur-Formate

Die Grundlage für ein automatisiertes Umblättersystem bildet die digitale Repräsentation der Partitur. Sofern diese nicht bereits in einem maschinenlesbaren Format vorliegt, müssen die Informationen mittels OMR-Verfahren aus Bilddaten (z. B. Scans von Notenblättern) extrahiert werden.

2.1.1 Definition und Abgrenzung

Nach Calvo-Zaragoza et al. wird OMR als Forschungsfeld definiert, welches das computergestützte Auslesen von Musiknotation in Dokumenten untersucht [7]. Obwohl Ähnlichkeiten zur optischen Zeichenerkennung (engl. Optical Character Recognition (OCR)) bestehen, weist OMR spezifische Herausforderungen auf: Die Notation ist stark kontextabhängig, weist eine zweidimensionale Struktur auf (vertikale Tonhöhe und horizontale Zeitachse) und beinhaltet Symbole mit stark variierenden Größenverhältnissen [7].

In der Literatur werden primär zwei Zielsetzungen der OMR unterschieden, welche die Struktur der nachgelagerten Verarbeitungsschritte maßgeblich definieren (siehe Abbildung 2.1) [7]:

1. **Rekonstruktion der vollständigen Notation:** Ziel ist die Erfassung des Layouts und der grafischen Details, um eine visuelle Reproduktion, etwa im MusicXML-Format, zu ermöglichen.
2. **Rückgewinnung der musikalischen Semantik:** Hierbei steht die Extraktion der reinen musikalischen Information (Tonhöhen, exakte Zeitwerte) im Vordergrund, welche häufig im Musical Instrument Digital Interface (MIDI)-Format repräsentiert wird.

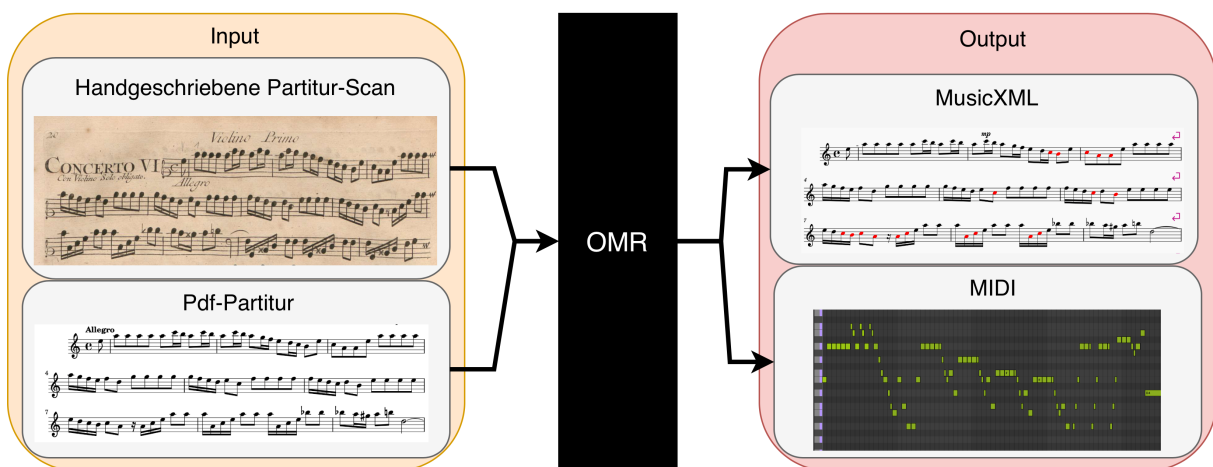


Abbildung 2.1: Schematische Darstellung des OMR-Prozesses

2.1.2 Digitale Formate: MusicXML und MIDI

Für den Datenaustausch zwischen Notensatzprogrammen hat sich MusicXML als Standard etabliert. Das von Michael Good entwickelte Format basiert auf Extensible Markup Language (XML) und ermöglicht die Speicherung sowohl der semantischen als auch der darstellungsbezogenen Musikdaten [8]. Im Gegensatz dazu dient das MIDI-Format der Übertragung von Steuerbefehlen und präzisen Zeitinformationen zwischen digitalen Systemen [9]. Während MIDI musikalische Parameter wie Tonhöhe und Rhythmik über exakte Zeitstempel hochpräzise erfasst und effizient für die Audiosynthese genutzt werden kann, fehlen ihm die für den klassischen Notensatz notwendigen Informationen über die grafische Anordnung (z. B. Layout, Seitenumbrüche oder komplexe strukturelle Notationselemente).

2.1.3 Datenverfügbarkeit und OMR-Software

Digitale Partiturdaten können über verschiedene Wege bezogen werden. Neben kommerziellen Quellen existieren Crowdsourcing-Plattformen und wissenschaftliche Datensätze. Das *MetaMIDI*-Dataset stellt eine umfangreiche Sammlung von MIDI-Dateien bereit, die für Forschungszwecke genutzt werden können [10]. Portale wie *MuseScore.com* [11] oder die *International Music Score Library Project (IMSLP)* [12] bieten zudem Zugriff auf eine Vielzahl von manuell erstellten oder mittels OMR konvertierten Dokumenten.

Für die automatisierte Konversion von Portable Document Format (PDF)-Dokumenten in MusicXML-Dateien stellt *Audiveris* [13] eine verbreitete Open-Source-Lösung dar. Dennoch bleibt die Genauigkeit von OMR-Systemen ein limitierender Faktor. Nach [14] beeinflussen drei Faktoren die Wahl und den Erfolg eines OMR-Verfahrens:

- Die Art der Notation (z. B. moderne westliche Notation vs. Tabulaturen),
- das Transkriptionsformat (handschriftlich vs. digital erstellt),
- die applikationsspezifischen Anforderungen an die Genauigkeit (hohe Präzision für Playback vs. moderate Präzision für die Musik-Synchronisation).

Untersuchungen an diversen OMR-Programmen (wie *SharpEye*, *SmartScore* oder *PhotoScore*) zeigen, dass verschiedene Systeme bei identischen Quellen divergierende Fehlerprofile aufweisen. Während einige Engines Schwierigkeiten bei der Erkennung von Notenhälsen haben, versagen andere bei spezifischen Tonhöhen oder der Zuordnung von Textelementen [15]. Ein generelles, systemübergreifendes Problem der optischen Erkennung ist die fehleranfällige Interpretation von Dynamikangaben. In nachgelagerten Audiosignalverarbeitungsschritten erfordert dies häufig den Einsatz algorithmischer Kompensationsverfahren, wie beispielsweise Vektornormierungen, um Lautstärkechwankungen als Fehlerquelle zu eliminieren.

2.2 Audiosignalverarbeitung und Merkmalsextraktion

Rohaudiosignale liegen im Zeitbereich als dichte Abfolge einzelner Abtastwerte vor und enthalten keine direkte Repräsentation der harmonischen oder spektralen Struktur, die für einen maschinellen Abgleich von Musikdaten erforderlich ist. Daher ist eine Transformation in den Frequenzbereich sowie eine anschließende Merkmalsextraktion notwendig.

2.2.1 Zeit-Frequenz-Transformationen

Die grundlegende Methode zur Analyse des Frequenzspektrums eines zeitvarianten Signals ist die Short-Time Fourier Transform (STFT). Dabei wird das Signal $x[n]$ durch eine Fensterfunktion $w[n]$ in kurze, überlappende Segmente (engl. Frames) unterteilt. Für jedes Fenster wird die Diskrete Fourier-Transformation (DFT) berechnet. Mathematisch ist die zeitdiskrete STFT definiert als [16]:

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n]w[n-m]e^{-j\omega n} \quad (2.1)$$

Die STFT liefert eine Matrix $X(m, \omega)$, deren Einträge die Amplitude und Phase der Frequenzkomponenten ω zum Zeitpunkt m repräsentieren. Die grafische oder mathematische Darstellung dieser zeitlichen Abfolge von Frequenzspektren wird als Spektrogramm bezeichnet.

Ein physikalisches Merkmal der STFT ist die lineare Frequenzauflösung. Das analysierte Frequenzspektrum wird hierbei in diskrete Intervalle, sogenannte Frequenz-Bins, unterteilt. Die Bandbreite eines einzelnen Bins (Δf) berechnet sich aus dem Quotienten der Abtastrate des Audiosignals (f_s) und der gewählten Fenstergröße (N_{FFT}) in Samples: $\Delta f = \frac{f_s}{N_{FFT}}$. Der Frequenzabstand zwischen zwei benachbarten Bins bleibt folglich über das gesamte Spektrum konstant.

Im Gegensatz zur STFT basiert die Constant-Q Transform (CQT) auf einer logarithmischen Frequenzskala. Die Mittenfrequenzen f_k der Bins sind geometrisch angeordnet und korrelieren direkt mit der westlichen Musikskala, bei der Halbtöne ebenfalls logarithmisch verteilt sind [17]. Das Verhältnis aus Mittenfrequenz und Bandbreite (der Q-Faktor) bleibt dabei für alle Bins konstant [18]:

$$Q = \frac{f_k}{\Delta f_k} = \text{const.} \quad (2.2)$$

Dies führt zu einer variablen Fenstergröße im Zeitbereich: Tiefe Frequenzen werden mit längeren Fenstern analysiert (hohe Frequenzauflösung), hohe Frequenzen mit kürzeren Fenstern (hohe Zeitaufklärung).

2.2.2 Spezifische Merkmalsvektoren für Audiosignale

Für die computergestützte Analyse von Musik werden aus den berechneten Spektrogrammen reduzierte Merkmalsvektoren extrahiert. Ein etabliertes Verfahren ist die

Berechnung von Chroma-Features. Dabei wird das Frequenzspektrum auf die 12 Tonklassen der chromatischen Tonleiter abgebildet, indem die spektrale Energie aller Oktaven für den jeweiligen Grundton aufsummiert wird. Dieser Zuordnungsprozess ist in Abbildung 2.2 schematisch dargestellt [19]. Das resultierende 12-dimensionale Array stellt die harmonische Struktur eines Audio-Frames dar. Durch die Aufsummierung über alle Oktaven besitzen Chroma-Features die Eigenschaft der Oktavinvarianz.

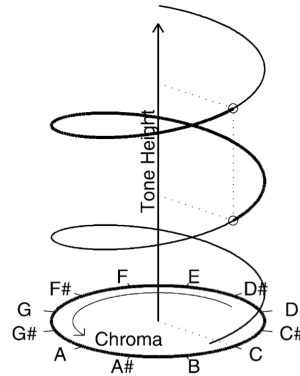


Abbildung 2.2: Zuordnung der Frequenz-Bins zu den 12 Pitch-Classes eines Chroma-Vektors [19]

Als komplementäres Merkmal werden in der Signalverarbeitung Mel-Frequency Cepstral Coefficients (MFCCs) genutzt, um das Timbre (die spektrale Hüllkurve) eines Audiosignals abzubilden [20]. Im musikalischen Kontext dienen MFCCs unter anderem zur Unterscheidung von Instrumenten oder Audioquellen [21].

2.2.3 Distanzmaße in der Signalverarbeitung

Um Merkmalsvektoren automatisiert vergleichen zu können, werden mathematische Distanzmetriken benötigt. Ein Standardverfahren ist die Euklidische Distanz $d_{euc}(u, v)$, die den direkten geometrischen Abstand zwischen zwei Vektoren u, v im Vektorraum beschreibt [22]:

$$d_{euc}(u, v) = \sqrt{\sum_{k=1}^K (u_k - v_k)^2} = \|u - v\|_2 \quad (2.3)$$

Die Euklidische Distanz ist sensitiv gegenüber der absoluten Größe der Vektoren. In der Audiosignalverarbeitung resultieren Lautstärkeunterschiede in einer Skalierung der Vektorlänge, was den Euklidischen Abstand vergrößert, selbst wenn der harmonische Inhalt identisch ist.

Die Kosinus-Distanz $d_{\cos}(u, v)$ eliminiert diesen Effekt, indem sie ausschließlich den Winkel θ zwischen zwei Vektoren betrachtet. Sie definiert sich über das Skalarprodukt der Vektoren, normiert durch deren jeweilige Längen [22]:

$$d_{\cos}(u, v) = 1 - \cos(\theta) = 1 - \frac{u \cdot v}{\|u\|_2 \|v\|_2} \quad (2.4)$$

Wird auf die Vektoren vor der Distanzberechnung eine L_2 -Normierung angewendet, weisen alle Vektoren die Einheitslänge auf (die Norm $\|v\|_2$ beträgt demnach 1) [23]. Bei normierten Vektoren reduziert sich die Berechnung der Kosinus-Distanz auf ein Skalarprodukt gefolgt von einer einfachen Subtraktion:

$$d_{\cos}(u_{\text{norm}}, v_{\text{norm}}) = 1 - (u_{\text{norm}} \cdot v_{\text{norm}}) \quad (2.5)$$

Dieses Verfahren gewährleistet mathematische Amplitudeninvarianz. Der geometrische Unterschied in der Berechnung zwischen der längenabhängigen Euklidischen Distanz und der Kosinus-Distanz bei L_2 -normierten Vektoren ist in Abbildung 2.3 visualisiert. Für das Tracking von Partituren bedeutet diese Amplitudeninvarianz, dass Schwankungen in der Ausführungslautstärke oder fehlerhaft erfasste Dynamikangaben durch OMR die Distanzberechnung nicht beeinflussen.

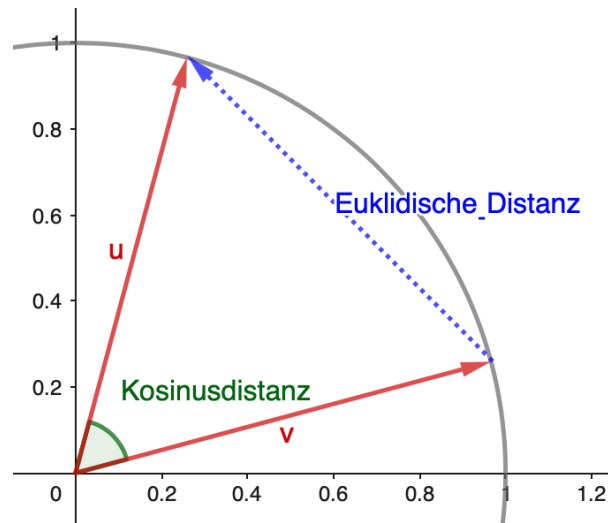


Abbildung 2.3: Geometrischer Vergleich zwischen Euklidischer Distanz und Kosinus-Distanz bei zweidimensionalen, L_2 -normierten Vektoren.

2.3 Score Following

Die automatisierte, zeitliche Synchronisation einer musikalischen Darbietung mit einer maschinenlesbaren Repräsentation der zugehörigen Partitur in Echtzeit wird in der Literatur als Score Following bezeichnet. Der Ursprung dieses Forschungsfeldes liegt in den frühen 1980er Jahren, primär motiviert durch die Entwicklung interaktiver Computerbegleitsysteme [24]. Moderne Systeme analysieren das kontinuierliche Audiosignal, extrahieren spezifische spektrale oder harmonische Merkmale und ordnen diese fortlaufend den korrespondierenden Positionen in der Referenzpartitur zu [24]. Zur Lösung dieses Zuordnungsproblems haben sich im Laufe der Zeit verschiedene methodische Paradigmen etabliert, die sich in ihrer Systemarchitektur und den zugrunde liegenden Tracking-Algorithmen unterscheiden.

2.3.1 Methodische Paradigmen

Hidden Markov Models (HMMs) Probabilistische Ansätze modellieren den Abgleichprozess häufig durch Hidden Markov Models (HMMs). In diesem Paradigma wird die Partitur als eine deterministische Abfolge verborgener Zustände (engl. hidden states) betrachtet. Das akustische Live-Signal fungiert dabei als die messbare Ausprägung dieser verborgenen Zustände [25]. HMMs erlauben es, die definierte Struktur der Partitur – also die bekannte Abfolge der Noten – direkt als mathematisches Regelwerk in das Modell einzubetten. Darüber hinaus modellieren sie Abweichungen in der Performance (wie Spielfehler oder Tempoänderungen) über Wahrscheinlichkeitsverteilungen. Der Algorithmus berechnet fortlaufend den wahrscheinlichsten aktuellen Zustand in der Partitur basierend auf den bisherigen Audiobeobachtungen.

Maschinelles Lernen und Deep Learning Ein neueres Forschungsparadigma bilden Verfahren der künstlichen Intelligenz, insbesondere tiefe neuronale Netze (Deep Learning). Diese Ansätze formulieren den Zuordnungsprozess als End-to-End-Problem. Erste Architekturen nutzen beispielsweise faltungsneuronale Netze (engl. Convolutional Neural Networks), um auf der STFT basierende Audiospektrogramme direkt mit Pixelkoordinaten in unstrukturierten Partiturbildern abzugleichen [26]. Darauf aufbauend untersuchen weiterführende Arbeiten den Einsatz von Reinforcement Learning. Hierbei erlernen Agenten das synchrone Verfolgen der Partitur durch kontinuierliche Interaktion in einer simulierten Umgebung [27]. Ein limitierender Faktor dieser datengetriebenen Methoden ist der hohe Bedarf an annotierten Trainingsdaten. Um eine Generalisierungsfähigkeit über verschiedene Instrumente, Tempi und akustische

Bedingungen hinweg zu gewährleisten, müssen umfangreiche Datensätze präzise ausgerichteter Audio-Partitur-Paare vorliegen [27].

Dynamic Time Warping (DTW) Als Alternative zu rein statistischen oder datengetriebenen Modellen kommen deterministische Distanzminimierungsalgorithmen zum Einsatz. Das klassische Dynamic Time Warping (DTW) berechnet die optimale zeitliche Übereinstimmung zwischen zwei Sequenzen, die Nichtlinearitäten in der Ausführungsgeschwindigkeit aufweisen [28]. Da der ursprüngliche Algorithmus die vollständige Kenntnis beider Sequenzen voraussetzt, wurde für Echtzeitanwendungen das Online Time Warping (ODTW) entwickelt. Dieser Ansatz berechnet den optimalen Pfad inkrementell und evaluiert eintreffende Audio-Frames kontinuierlich gegen die vorberechnete Referenzsequenz [29]. Durch diese algorithmische Modifikation ermöglicht OTDW ein ressourceneffizientes Tracking ohne die Notwendigkeit eines initialen Modelltrainings.

2.3.2 Vergleich der Literaturansätze

Die beschriebenen Paradigmen unterscheiden sich nicht nur im verwendeten Tracking-Algorithmus, sondern auch in der Vorverarbeitung der Audiosignale. Abbildung 2.4 ordnet die Kernpublikationen des Forschungsfeldes historisch ein und verdeutlicht, welche Kombinationen aus Algorithmus und Signaltransformation zu welcher Zeit den Stand der Technik definierten.

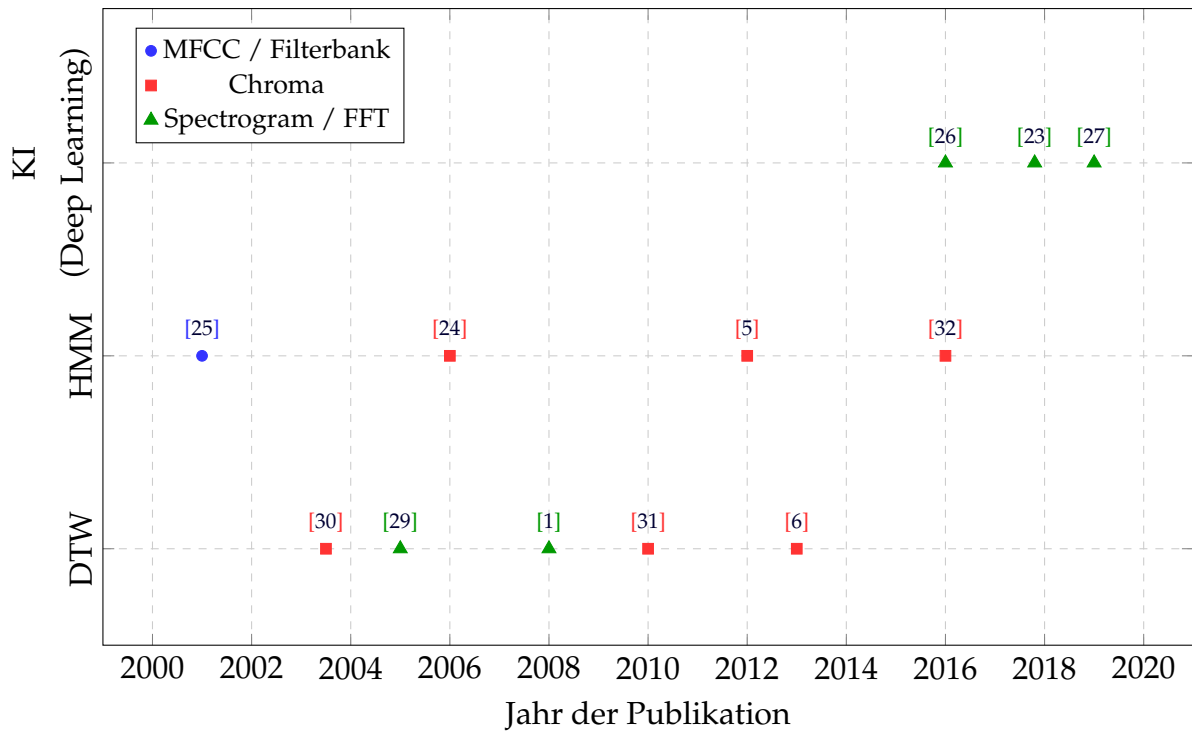


Abbildung 2.4: Zeitliche Entwicklung der Algorithmen (DTW, HMM, KI) in Abhängigkeit vom verwendeten Input-Feature.

2.4 Dynamic Time Warping (DTW)

Der Kern eines abstands-basierten Score-Following-Systems ist der Algorithmus zur zeitlichen Ausrichtung des kontinuierlichen Audiosignals mit der Referenz. Das DTW stellt ein etabliertes Verfahren der dynamischen Programmierung dar, um die optimale Synchronisation zwischen zwei zeitabhängigen Sequenzen mit nicht-linearen Geschwindigkeitsabweichungen zu ermitteln [33].

2.4.1 Klassisches Offline-DTW

In der ursprünglichen, für die Spracherkennung entwickelten Form vergleicht das DTW eine Testsequenz mit einer Referenzsequenz über deren gesamte Länge [33]. Gegeben seien zwei Vektorsequenzen Y (Live-Signal) und X (Referenz). Zunächst wird eine lokale Kostenmatrix $C(i, j)$ berechnet. Die Einträge der Matrix quantifizieren die paarweise Distanz (beispielsweise über die Kosinus-Distanz) zwischen den Vektoren y_i und x_j [30].

Ziel des Algorithmus ist die Bestimmung eines minimalen Pfades (engl. warping path) durch diese Matrix, der den Startpunkt $(1, 1)$ und den Endpunkt (N, M) verbindet. Die Koordinaten des Endpunkts entsprechen dabei den jeweiligen Gesamtlängen der beiden Vektorsequenzen. Dieser Pfad definiert die resultierende nicht-lineare zeitliche Ausrichtung der beiden Sequenzen (siehe Abbildung 2.5). Hierzu wird eine kumulierte Kostenmatrix $D(i, j)$ rekursiv berechnet. Die akkumulierten Kosten an einer Position (i, j) ergeben sich aus der Summe der lokalen Kosten $C(i, j)$ und dem Minimum der zulässigen Vorgängerzellen.

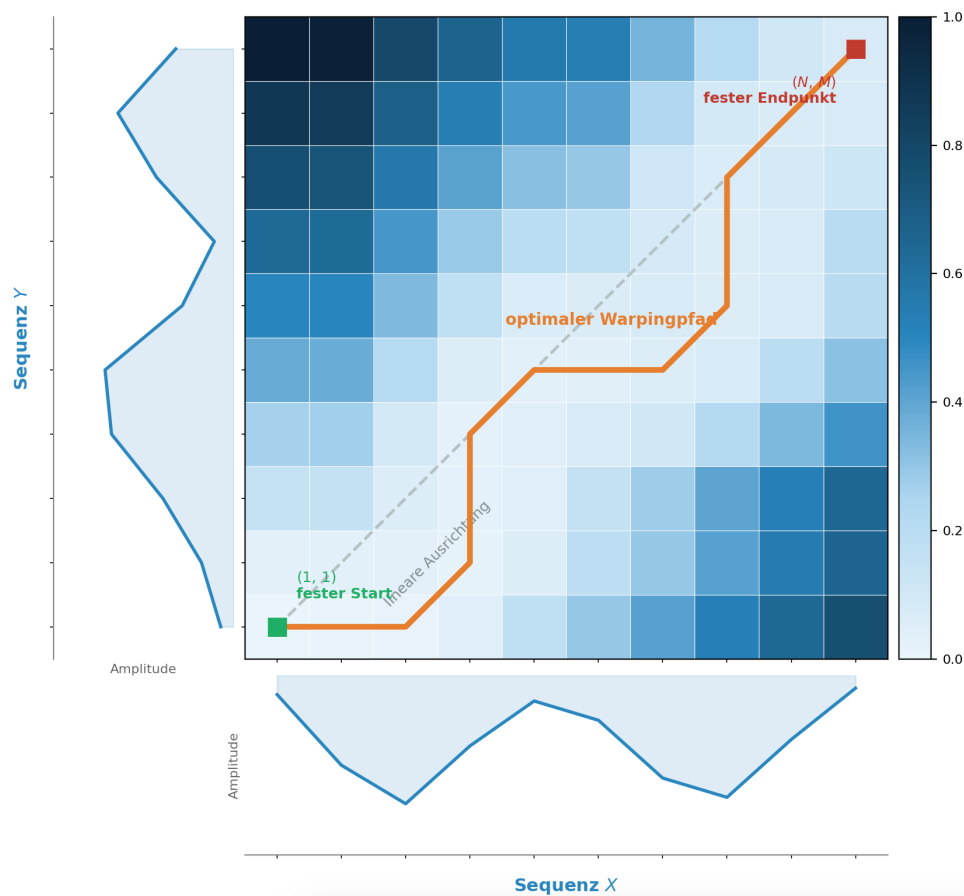


Abbildung 2.5: Visualisierungsbeispiel des Dynamic Time Warping-Algorithmus.

Das klassische Offline-DTW erfordert zwingend die vollständige Vorabkenntnis beider Sequenzen. Die Berechnung der gesamten Matrix skaliert quadratisch mit einer zeitlichen und räumlichen Komplexität von $\mathcal{O}(N \cdot M)$ [30]. Für Live-Anwendungen, bei denen eine Sequenz als unbegrenzter Audiodatenstrom vorliegt, ist dieses Paradigma folglich nicht anwendbar.

2.4.2 Subsequence Dynamic Time Warping

Eine Erweiterung des klassischen Verfahrens stellt das Subsequence-DTW dar. Während das globale DTW die gesamte Testsequenz gegen die gesamte Referenzsequenz ausrichtet, ermöglicht das Subsequence-DTW das Auffinden einer kurzen Suchsequenz innerhalb eines wesentlich längeren Referenzsignals [34]. In einem Partitur-Tracking-System ist dies die mathematische Grundlage für den Recovery-Mechanismus: Tritt ein Positionsverlust ein, wird ein Puffer der letzten Audio-Frames genutzt, um die aktuelle Position in der vollständigen Partitur erneut zu identifizieren [35].

Der entscheidende Unterschied zum globalen Ansatz liegt in der Initialisierung der kumulierten Kostenmatrix $D(i, j)$. Um einen Einstieg an einer beliebigen Stelle in der Referenzsequenz zu ermöglichen, wird die erste Zeile der Matrix so definiert, dass kein initialer Distanzwert akkumuliert wird. Nach [34] gilt für eine Suchsequenz Y der Länge N und eine Referenz X der Länge M :

$$D(0, j) = 0 \quad \text{für } j \in [0, M] \quad (2.6)$$

sowie

$$D(i, 0) = \infty \quad \text{für } i \in [1, N] \quad (2.7)$$

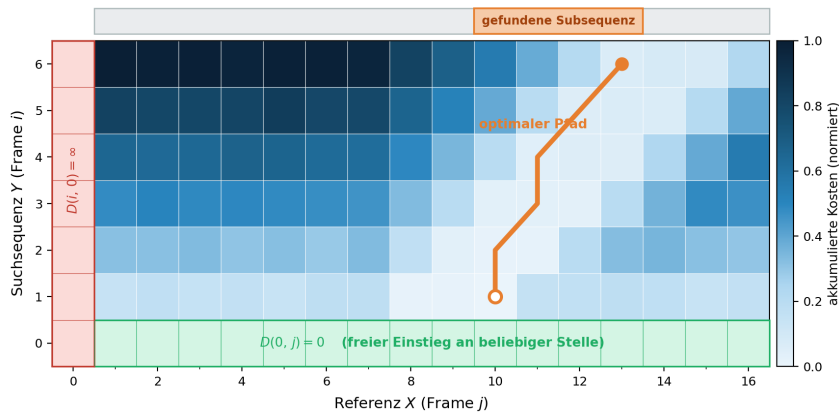


Abbildung 2.6: Visualisierungsbeispiel des Subsequence Dynamic Time Warping-Algorithmus.

Durch diese Randbedingungen kann der optimale Pfad an jedem beliebigen Punkt j der Referenz beginnen, ohne durch die Distanzwerte früherer Frames beeinflusst zu werden. Das Ende der Subsequenz wird durch das Minimum in der letzten Zeile der Matrix $D(N, j)$ bestimmt. Ausgehend von diesem Endpunkt lässt sich die exakte zeitliche Ausrichtung durch ein Backtracking bis zum Startpunkt auf der Referenzachse ermitteln (siehe Abbildung 2.6). Dieser Ansatz erlaubt eine robuste Re-Synchronisation, da er unabhängig von der vorherigen Tracking-Historie agiert.

2.4.3 Online Dynamic Time Warping (ODTW)

Um das Verfahren echtzeitfähig zu machen, erfolgt die algorithmische Modifikation zum OTW [29]. Anstatt die vollständige Kostenmatrix vorab zu berechnen, arbeitet das OTW inkrementell. Mit jedem neu eintreffenden Audio-Frame wird lediglich ein definierter Teilbereich (Suchkorridor) der Matrix aktualisiert. Der Pfad wird somit schrittweise entlang der Zeitachse des Live-Signals evaluiert, was die Rechenlast auf einen konstanten Wert pro Frame reduziert [28].

Pfad-Optionen und die Kostenmatrix

Die Toleranz des Trackers gegenüber variierenden Ausführungsgeschwindigkeiten wird algorithmisch durch die zulässigen Pfad-Optionen (engl. *step conditions*) definiert. Im Gegensatz zum klassischen Verfahren wird die Kostenmatrix beim OTW nicht vollständig im Voraus berechnet. Vielmehr erfolgt die Evaluierung inkrementell: Die Dimensionen der Matrix korrespondieren mit den beiden abgeglichenen Zeitreihen. Dabei definiert i den diskreten Zeitindex des fortlaufenden Live-Signals und j den Positionsindex innerhalb der vorberechneten Referenzpartitur. Mit jedem neu eintreffenden Live-Frame am Index i wird der Pfad relativ zur Referenz fortgeschrieben. Für diese schrittweise Berechnung der akkumulierten Kosten ergeben sich typischerweise drei primäre Übergangsmöglichkeiten [1], [31]:

- **Wait** ($i - 1, j$): Das System aktualisiert das eintreffende Live-Signal, verharret jedoch auf der aktuellen Partiturposition. Dieser Fall tritt ein, wenn das Live-Tempo langsamer ist als die Referenz.
- **Step** ($i - 1, j - 1$): Der diagonale Fortschritt. Sowohl das Live-Signal als auch die Referenzpartitur schreiten gleichmäßig voran. Dies ist der Standardfall bei synchronem Tempo.
- **Skip** ($i - 1, j - 2$): Während das Live-Signal um einen Frame fortschreitet, überspringt das System einen Frame der Referenz. Diese Option kompensiert Darbietungen, die signifikant schneller sind als die Vorlage.

Obwohl die mathematische Formulierung eine vollständige zweidimensionale Matrix aufspannt, erfordert die praktische Implementierung für Echtzeitsysteme nicht die Speicherung der gesamten Historie. Es ist rechen- und speichereffizient, lediglich die akkumulierten Kosten des vorherigen Frames ($i - 1$) vorzuhalten, um den aktuellen Zustandsvektor zum Zeitpunkt i zu berechnen. Dieser Update-Schritt sowie die

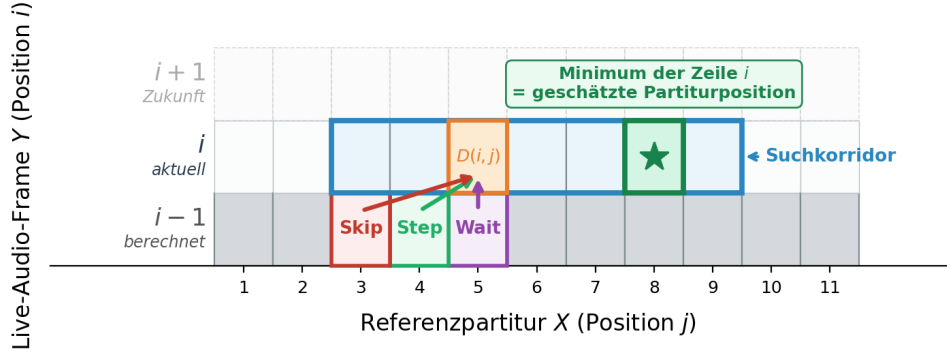


Abbildung 2.7: Visualisierungsbeispiel eines ODTW-Schritts.

geometrische Zuordnung der drei Pfad-Optionen sind in Abbildung 2.7 veranschaulicht. Die rekursive Berechnung der akkumulierten Kosten $D(i, j)$ für eine Zelle an der Referenzposition j erfolgt somit über die Additionsregel:

$$D(i, j) = C(i, j) + \min \begin{cases} D(i-1, j) + w_{wait} \\ D(i-1, j-1) + w_{step} \\ D(i-1, j-2) + w_{skip} \end{cases} \quad (2.8)$$

Hierbei quantifiziert die lokale Kostenfunktion $C(i, j)$ die Vektordistanz zwischen dem Merkmal des eintreffenden Live-Frames i und dem Merkmal der Referenz an der Position j . Den Pfad-Optionen können durch die Parameter w_{wait} , w_{step} und w_{skip} separate Strafgewichte (Penalties) zugewiesen werden. Dadurch lässt sich das Verhalten des Trackers deterministisch steuern, um beispielsweise einen stetigen Vorwärtsfluss (*Step*) gegenüber einem Stillstand (*Wait*) zu begünstigen.

Das Damping-Prinzip

Eine mathematische Konsequenz der kontinuierlichen Pfadberechnung ist das stetige Wachstum der akkumulierten Kosten. Da fortlaufend neue Distanzwerte addiert werden, summieren sich diese Werte über einen längeren Zeitraum zu immer größeren Beträgen auf. Dies kann einerseits zu numerischen Überläufen führen. Andererseits bewirken sehr hohe kumulierte Kosten, dass der Algorithmus träge auf aktuelle Veränderungen reagiert, da die historischen Abweichungen das aktuelle Distanzmaß dominieren [28].

Das Damping-Prinzip löst dieses Problem durch eine kontinuierliche Dämpfung der Kostenfunktion. Nach der Evaluierung eines Frames werden die akkumulierten Kos-

ten mit einem Dämpfungsfaktor d (wobei $0 < d < 1$) multipliziert [31]. Mathematisch bewirkt dies einen exponentiellen Verfall vergangener Fehler. Neuere Audio-Beobachtungen erhalten ein höheres relatives Gewicht, wodurch die Sensitivität des Trackers für kurzfristige Tempo- und Positionsänderungen erhalten bleibt.

3 Methodik und Systemarchitektur

Dieses Kapitel beschreibt die methodischen Entscheidungen und die architektonische Umsetzung des entwickelten Systems zur automatisierten Partiturverfolgung. Zunächst werden die Anforderungen an das System spezifiziert. Darauf aufbauend wird die Strukturierung in eine vorbereitende Offline- und eine ausführende Echtzeit-Komponente erläutert. Im weiteren Verlauf werden die algorithmischen Grundlagen begründet, die Vorverarbeitungsschritte zur Generierung der Referenzdaten beschrieben und die asynchrone Softwarearchitektur des Live-Trackers im Detail dargestellt.

3.1 Anforderungsanalyse

Vor der detaillierten Ausarbeitung der Systemarchitektur werden die funktionalen und nicht-funktionalen Anforderungen an das System definiert. Diese Spezifikationen bilden die objektive Bemessungsgrundlage für die anschließenden algorithmischen und strukturellen Designentscheidungen.

3.1.1 Funktionale Anforderungen

Die funktionalen Anforderungen spezifizieren die zwingend erforderlichen Prozesse, die das System zur Zielerreichung ausführen muss:

- F1 – Formatkompatibilität:** Das System muss in der Lage sein, sowohl herkömmliche PDF-Partituren als auch native MusicXML-Dateien zu verarbeiten. Bei Vorliegen von Bilddaten (PDF) wird eine vorgelagerte Verarbeitung mittels OMR vorausgesetzt, um die Daten in ein maschinenlesbares Format zu überführen.
- F2 – Automatisiertes Blättern:** Die Generierung und Übertragung eines MIDI-Signals zur Auslösung des Seitenwechsels erfolgt vollständig autonom. Zur Aufrechterhaltung des Leseflusses ist das Auslösesignal mit einem definierten Vorlauf (beispielsweise eine Sekunde vor dem tatsächlichen Seitenende) zu senden.

F3 – Re-Synchronisation (Recovery): Im Falle von starken Abweichungen, musikalischen Wiederholungen oder ausgeprägten Spielfehlern muss das System die aktuelle Position innerhalb der Partitur selbstständig neu ermitteln können, um einen dauerhaften Verlust der Verfolgung zu verhindern.

3.1.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen definieren die Qualitätskriterien und technischen Rahmenbedingungen der Systemausführung:

N1 – Echtzeitfähigkeit: Die Latenz zwischen dem Erfassen des akustischen Eingangssignals und der algorithmischen Positionsbestimmung muss hinreichend gering sein, um das Blättern synchron zur Darbietung auszuführen. Dies erfordert die Verarbeitung von Analyse-Frames im unteren Millisekunden-Bereich.

N2 – Robustheit gegen Störgeräusche: Das Systemverhalten muss eine hohe Toleranz gegenüber Raumrauschen aufweisen. Ebenso dürfen fehlerhaft erfasste Dynamikangaben durch die vorangegangene OMR-Analyse das Tracking-Verhalten nicht negativ beeinflussen.

N3 – Tempoinvarianz: Der Tracking-Algorithmus muss sich an das individuelle und zeitlich variable Spieltempo anpassen. Eine manuelle Vorab-Modifikation des Referenz-Tempos ist auszuschließen.

N4 – Ressourceneffizienz: Die algorithmische Komplexität, insbesondere während der rechenintensiven Re-Synchronisation, ist zu minimieren. Die Rechenlast muss so skaliert sein, dass eine zukünftige Portierung auf leistungsschwächere Hardwarearchitekturen (Edge-Devices) realisierbar bleibt.

3.2 Gesamtkonzept und methodische Entscheidungen

Die vollständige Quellcode-Implementierung des in dieser Arbeit entwickelten Systems ist als Open-Source-Projekt in einem GitHub-Repository zugänglich¹.

Zur Erfüllung der definierten Anforderungen basiert das architektonische Fundament des Systems auf einer strikten Trennung in zwei modulare Verarbeitungsebenen, wie in Abbildung 3.1 dargestellt: Die vorbereitende Offline-Score-Pipeline und den ausführenden Live-Page-Turner. Diese Trennung stellt sicher, dass ressourcenintensive

¹ <https://github.com/SamsiGg/2.-Studienarbeit-Smarter-Page-Turner.git>

Datenaufbereitungsprozesse im Vorfeld abgeschlossen werden. Dadurch werden die Anforderungen an Echtzeitfähigkeit (N1) und Ressourceneffizienz (N4) in der Laufzeitumgebung erfüllt. Die genaue Funktionsweise der Offline-Datenaufbereitung wird im nachfolgenden Abschnitt 3.3 detailliert erläutert, während die architektonische Umsetzung der Echtzeit-Komponente Gegenstand von Abschnitt 3.4 ist.

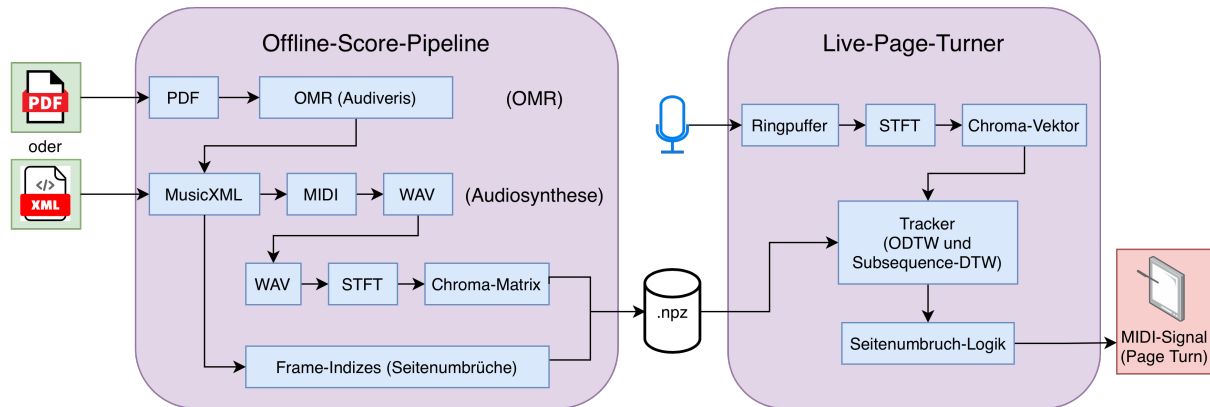


Abbildung 3.1: Zweigeteilte Systemarchitektur: Datenfluss von der Offline-Score-Pipeline zur Echtzeit-Tracking-Komponente.

3.2.1 Wahl des Tracking-Algorithmus

Für die zeitliche Ausrichtung des kontinuierlichen Audiosignals mit der Referenzpartitur wird das Online Dynamic Time Warping (ODTW) anstelle von probabilistischen Modellen wie HMMs oder neuronalen Netzen verwendet. Die Entscheidung für OTDW begründet sich durch die deterministische Transparenz des Algorithmus. Im Gegensatz zu maschinellen Lernverfahren, die umfangreiche, annotierte Datensätze für das Training erfordern, ermöglicht das distanzbasierte OTDW eine Zero-Shot-Fähigkeit. Das System ist somit in der Lage, beliebige Partituren ohne vorheriges, modellspezifisches Training abzugleichen. Zudem sichert die nicht-lineare Pfadsuche des OTDW die Erfüllung der Tempoinvarianz (N3).

3.2.2 Wahl der Signaltransformation

Zur Überführung des Audiosignals in den Frequenzbereich implementiert das System die Short-Time Fourier Transform (STFT) anstelle der Constant-Q Transform (CQT). Während die CQT musikalische Frequenzen zwar vorteilhaft logarithmisch abbildet, lässt sich die STFT direkt und hocheffizient über die Fast Fourier Transform (FFT) berechnen. Diese höhere Recheneffizienz minimiert die Central Processing Unit (CPU)-Auslastung und ist eine direkte Maßnahme zur Einhaltung der Anforderung N4.

3.3 Die Score-Pipeline (Offline-Vorverarbeitung)

Die Score-Pipeline konvertiert eine vorliegende Partitur in ein kompaktes, maschinenlesbares Datenformat, das als Referenz für das Live-Tracking dient. Der Eingabeprozess des Systems ist dual ausgelegt und verarbeitet PDF-Dokumente sowie native MusicXML-Dateien (F1). Liegt die Partitur ausschließlich als PDF-Dokument vor, wird dieses zunächst über die Optical Music Recognition (OMR) mittels des Open-Source-Programms Audiveris [13] verarbeitet. Die programmatische Steuerung und Ausführung dieses Konvertierungsschritts ist in der Quellcodedatei `omr.py` implementiert. Die OMR-Engine extrahiert die grafisch dargestellten Notenwerte, Taktstriche und Seitenumbruch-Informationen und exportiert diese als MusicXML-Datei. Ab diesem Punkt konvergieren beide Eingabepfade, und die weitere Verarbeitung basiert einheitlich auf dem MusicXML-Format.

3.3.1 Audiosynthese und Referenz-Chroma

Die generierte oder nativ vorliegende MusicXML-Datei wird in das MIDI-Format überführt und anschließend mittels eines Software-Synthesizers in eine unkomprimierte WAV-Audiodatei synthetisiert. Die Vorgänge der Audiosynthese, der nachfolgenden Rauschmodellierung sowie der finalen Extraktion der Chroma-Matrix werden systemtechnisch gebündelt in der Methode `build_chroma()` innerhalb der Datei `chroma_builder.py` ausgeführt.

Die Entscheidung für eine vollständige Audiosynthese basiert auf empirischen Voruntersuchungen, in denen drei Ansätze zur Generierung der Chroma-Referenz evaluiert wurden: (1) die isolierte Zuweisung des Grundtons (Aktivierung ausschließlich der entsprechenden Pitch-Klasse), (2) die Anwendung einer festgelegten, tonhöhenabhängig verschobenen Oberton-Schablone des Zielinstruments sowie (3) die vollumfängliche Audiosynthese. Obwohl die vollständige Synthese den höchsten Vorab-Rechenaufwand erfordert, wurde dieses Verfahren implementiert. Es bildet das reale akustische Signalverhalten am exaktesten ab und maximiert dadurch die Robustheit des nachgelagerten Tracking-Algorithmus gegenüber spektralen Abweichungen im Live-Signal.

Im direkten Anschluss an die Synthese wird aus dem generierten Audiosignal die referenzierende Chroma-Matrix berechnet. Hierbei wird das Signal mittels STFT in den Frequenzbereich transformiert und anschließend auf die zwölf Halbtonklassen abgebildet. Das Ergebnis ist eine Matrix der Dimension $12 \times M$, wobei M der Anzahl

der zeitlichen Analyse-Frames der gesamten Partitur (Referenzsequenz) entspricht. Jeder Spaltenvektor repräsentiert die L2-normierte harmonische Energieverteilung eines spezifischen Zeitfensters. Abbildung 3.2 visualisiert einen solchen Ausschnitt der generierten Referenz-Matrix.

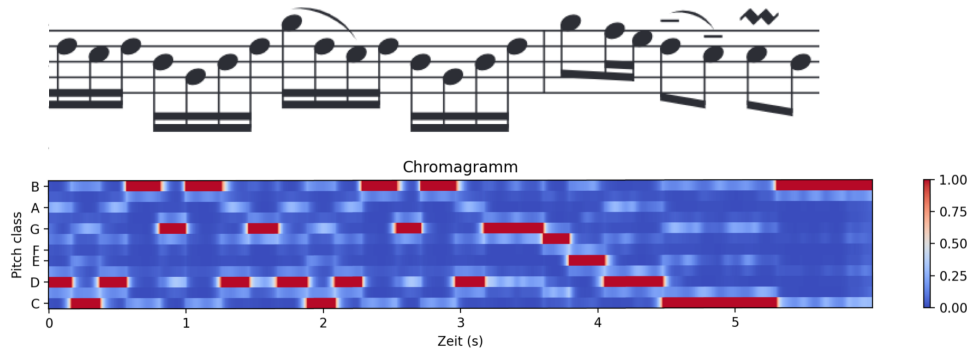


Abbildung 3.2: Ausschnitt einer berechneten Chroma-Matrix aus dem synthetisierten Referenz-Audio.

3.3.2 Rauschmodellierung

Ein systembedingtes Merkmal der Software-Synthese ist die Generierung von absoluter Stille nach dem Abklingen eines Tons. Der Synthesizer produziert in den Pausen einen reinen Hall-Effekt ohne physikalisches Hintergrundrauschen. Wird dieses Signal durch Normierungsverfahren skaliert, entsteht eine künstliche Merkmals-Signatur: Der leise, ausklingende Hall wird mathematisch auf die gleiche Vektorlänge wie ein voll gespielter Ton gestreckt, was ein Chroma-Profil erzeugt, das in der Realität nicht existiert. Da ein Mikrofon in Spielpausen stets Raumrauschen erfasst, führt diese Diskrepanz zu einer Fehlerakkumulation im Tracker.

Zur Erfüllung der Anforderung N2 (Robustheit gegen Störgeräusche) wird die physikalische Realität algorithmisch modelliert. Dem synthetisierten Audiosignal wird vor der Merkmalsextraktion additives weißes Rauschen mit einer definierten Standardabweichung ($\sigma = 0,004$) hinzugefügt. Dieser Wert approximiert den physikalischen Rauschpegel gängiger Mikrofone. Fällt die Signalamplitude des Synthesizers unter diesen Schwellwert, dominiert das eingefügte Rauschen. Das resultierende Merkmalsprofil der Referenz gleicht sich dadurch dem realen Audioinput an, was eine Fehlausrichtung in musikalischen Pausen verhindert. Dieser Effekt wird in Abbildung 3.3 veranschaulicht.

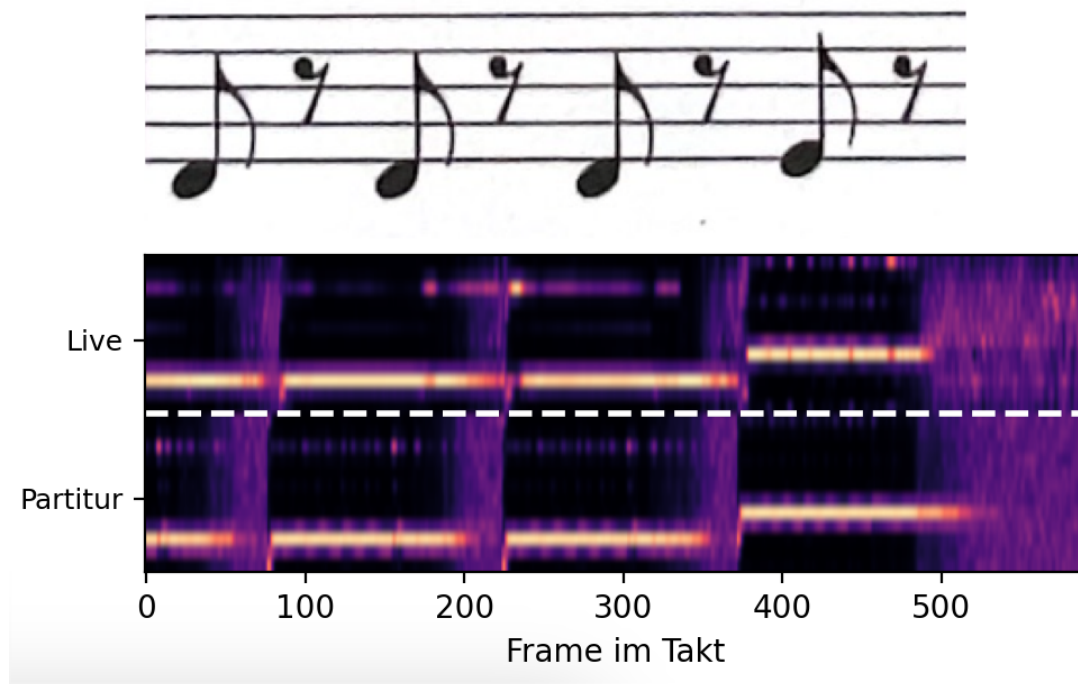


Abbildung 3.3: Vergleich von Chroma-Matrizen in musikalischen Pausen: Chroma-Signatur der durch das Rauschmodell korrigierten Referenz-Synthese (unten) und eine reale Mikrofonaufnahme (oben).

3.3.3 Metadaten-Mapping und Datenbündelung

Zusätzlich zur Generierung der Audiodaten extrahiert die Pipeline strukturelle Informationen aus der MusicXML-Datei. Räumliche Layout-Entscheidungen, wie Seitenumbrüche (gekennzeichnet durch XML-Tags wie `<print new-page="yes"/>`), werden identifiziert und unter Berücksichtigung des festgelegten Tempos in diskrete zeitliche Frame-Indizes übersetzt.

Abschließend werden alle aufbereiteten Daten gebündelt und in einem komprimierten NumPy-Archiv (.npz) gespeichert. Die programmatische Ausführung dieses Schreibvorgangs erfolgt durch die Methode `write_score_data()` in der Datei `score_writer.py`. Dieses Archiv dient als zentrale Referenzgrundlage für das Laufzeitsystem. Dabei wird technisch zwischen den funktional notwendigen Kernparametern für den Tracking-Prozess (F1 bis F3) und ergänzenden Metadaten für die grafische Benutzeroberfläche (engl. Graphical User Interface (GUI)) unterschieden. Während die Kernparameter die algorithmische Basis bilden, dienen die weiteren Informationen ausschließlich der visuellen Darstellung und Orientierung innerhalb der Applikation. Tabelle 3.1 spezifiziert die interne Struktur und die Datentypen des Datenpakets, wobei M die Gesamtzahl

der Analyse-Frames der Referenzsequenz und P die Anzahl der Seitenumbrüche beschreibt.

Tabelle 3.1: Struktur und Datentypen des .npz-Referenzarchivs, unterteilt in funktionale Kernkomponenten und UI-Metadaten.

Feldname	Datentyp	Beschreibung / Funktion
<i>Funktionale Kernkomponenten (Algorithmik)</i>		
chroma	float32 ($12 \times M$)	L2-normierte Referenz-Chroma-Matrix. Basis für den ODTW-Abgleich.
page_end_indices	int32 ($1 \times P$)	Array der Frame-Indizes für die Blätter-Logik (F2).
<i>Ergänzende Metadaten (User Interface)</i>		
bpm	int32	Referenz-Tempo für die Schlaganzeige in der GUI.
beats_per_measure	int32	Taktart-Information für die grafische Takt-Navigation.
musicxml_content	str	XML-Rohdaten zur optionalen Notendarstellung im Interface.

3.4 Architektur des Live-Systems (Echtzeit-Komponente)

Das Live-System verarbeitet die extrahierten Referenzdaten und synchronisiert diese fortlaufend mit dem akustischen Signal, um zum korrekten Zeitpunkt den Blättervorgang auszulösen.

3.4.1 Datenfluss

Der Signalweg zur Laufzeit ist unidirektional. Das System erfasst kontinuierlich das analoge Mikrofonsignal, digitalisiert dieses und extrahiert daraus schrittweise die Chroma-

Merkmalsvektoren. Die Ausführung dieser Extraktion wird durch die Klasse `ChromaExtractor` (Datei `chroma.py`) gesteuert. Die Vektoren werden anschließend an den ODTW-Tracker übergeben, dessen Berechnungslogik in der Datei `dtw.py` implementiert ist und welcher die aktuelle Position in der Referenzmatrix ermittelt. Erreicht die berechnete Position den Index eines zuvor definierten Seitenumbruchs abzüglich des definierten Vorlaufs, generiert das System ein Auslösesignal. Dieses Signal wird über das MIDI-Protokoll via Bluetooth übertragen (programmatisch abgebildet im Modul `ble_keyboard.py`), wodurch Anforderung F2 erfüllt wird.

Neben der regulären, fortlaufenden Positionsbestimmung beinhaltet der Tracking-Block einen Recovery-Mechanismus (Subsequence-DTW, ebenfalls verortet in `dtw.py`), um bei signifikanten Abweichungen oder Spielfehlern die Re-Synchronisation (F3) zu gewährleisten. Da das spezifische Algorithmus-Design, die Distanzberechnung und die genaue Funktionsweise dieses Recovery-Scans den analytischen Kern des Systems bilden, werden diese Komponenten losgelöst von der hier beschriebenen Systemarchitektur im nachfolgenden Kapitel 4 detailliert behandelt.

3.4.2 Datenpufferung über Sliding Windows

Für eine verzögerungsfreie Verarbeitung (N1) wird das kontinuierliche Audiosignal in diskrete Analyse-Frames überführt. Dies erfolgt durch die Implementierung eines Ringpuffers (Sliding Window), realisiert über die Klasse `AudioRingBuffer` in der Datei `chroma.py`, bei einer standardisierten Abtastrate f_s von 44.100 Hz. Das System liest das Audiosignal in Blöcken der Größe N_{hop} von 512 Samples ein und überführt diese in einen Puffer mit der Fenstergröße N_{FFT} von 4096 Samples. Bei jedem Schreibvorgang verfallen die ältesten 512 Samples, woraufhin über den aktualisierten Gesamtpuffer eine FFT durchgeführt wird.

Durch diese Blockgröße N_{hop} ergibt sich die zeitliche Update-Rate Δt des Trackers, in der neue Merkmalsvektoren berechnet werden:

$$\Delta t = \frac{N_{hop}}{f_s} = \frac{512}{44.100 \text{ Hz}} \approx 0,0116 \text{ s} = 11,6 \text{ ms} \quad (3.1)$$

Dieser Wert definiert das zeitliche Raster der nachgelagerten Verarbeitung: Alle 11,6 ms wird das durch die FFT berechnete Spektrum auf die zwölf Tonklassen abgebildet und zu einem aktuellen Chroma-Vektor weiterverarbeitet. Dies gewährleistet eine fortlaufende, hochfrequente Positionsbestimmung und erfüllt die Anforderung an eine echtzeitfähige Latenz. Die Festlegung der zugrunde liegenden Fenstergröße N_{FFT} von

4096 Samples erfordert jedoch eine methodische Abwägung zwischen Frequenz- und Zeitauflösung.

Einerseits definiert N_{FFT} die Frequenzauflösung Δf :

$$\Delta f = \frac{f_s}{N_{FFT}} = \frac{44.100 \text{ Hz}}{4096} \approx 10,77 \text{ Hz} \quad (3.2)$$

Diese feine Auflösung ist notwendig, um im tieffrequenten Bereich (wie bei den tiefsten Tönen der Violine G3 und G#3 mit einer physikalischen Differenz von $\Delta f \approx 11,65 \text{ Hz}$) die spektrale Überlappung in den Frequenz-Bins zu minimieren.

Andererseits bedingt die Fenstergröße von 4096 Samples bei gegebener Abtastrate eine absolute zeitliche Aufnahmedauer pro Frame von:

$$t_{win} = \frac{N_{FFT}}{f_s} = \frac{4096}{44.100 \text{ Hz}} \approx 92,88 \text{ ms} \quad (3.3)$$

Physikalisch resultiert diese Zeitspanne t_{win} von nahezu 100 ms in einer zeitlichen Unschärfe der Analyse. Töne, die eine kürzere Dauer als diese 93 ms aufweisen, können nicht mehr als isolierte musikalische Ereignisse abgebildet werden. Folgen Töne in einem Abstand von unter 200 ms aufeinander, entstehen zwangsläufig signifikante Signalüberlappungen innerhalb desselben Analysefensters, was eine Trennung der Chroma-Merkmale erschwert.

Zur Validierung dieses Trade-offs wird ein schnelles musikalisches Referenzbeispiel herangezogen: Bei einer Spielgeschwindigkeit von 100 Beats per Minute (BPM) weist ein Taktschlag eine Dauer von 600 ms auf. Eine Sechzehntelnote entspricht einem Viertel dieses Schlags und besitzt folglich eine theoretische Dauer von 150 ms. Da diese 150 ms die Analysefensterlänge t_{win} von $\approx 93 \text{ ms}$ überschreiten, verbleibt ein ausreichendes Zeitfenster, in dem der gespielte Ton dominant und ohne signifikante Überlappung durch den Nachbarton erfasst wird. Die Parametrisierung von 4096 Samples stellt somit ein empirisch belegtes Optimum dar, um eine ausreichende Frequenztrennung bei gleichzeitigem Erhalt der Rhythmusverfolgung zu gewährleisten.

3.4.3 Asynchrones Threading-Modell

Um eine Blockierung der Systemprozesse sowie Pufferüberläufe im Audio-Stream zu vermeiden, ist die Architektur in zwei separate Threads unterteilt. Die Instanziierung

und Verwaltung dieses asynchronen Modells erfolgt zentral in der Programmdatei `main.py`.

Der Worker-Thread ist exklusiv für die Ausführung zeitkritischer Operationen reserviert. Er verantwortet das kontinuierliche Auslesen des Audio-Interfaces, die Berechnung der FFT sowie die inkrementellen Rechenschritte des ODTW-Algorithmus. Der Main-Thread steuert die grafische Benutzeroberfläche sowie die ausgehende Bluetooth-Signalisierung.

Die Interprozesskommunikation erfolgt über eine threadsichere Warteschlange (Queue). Der Worker-Thread schreibt den aktuellen Systemzustand fortlaufend in diese Struktur. Der Main-Thread fragt die Queue in einem festen Intervall ab, verwirft veraltete Zustände und aktualisiert die Anzeige basierend auf dem aktuellsten Datenpunkt. Durch diese asynchrone Entkopplung wird sichergestellt, dass rechenintensive Algorithmus-Schritte, wie beispielsweise der notwendige Subsequence-Scan für die Re-Synchronisation (F3), die kontinuierliche Audio-Aufzeichnung nicht beeinträchtigen.

4 Algorithmus-Design: Echtzeit-Tracking und Recovery

Dieses Kapitel detailliert die algorithmische Umsetzung der Positionsbestimmung zur Laufzeit. Während Kapitel 3 die architektonische Struktur der Datenpufferung sowie die zeitliche Taktung der Analyse-Frames spezifiziert, steht hier die Verarbeitung dieser diskreten Datenströme im Fokus. Es wird dargelegt, wie die eintreffenden Chroma-Vektoren des Mikrofonsignals iterativ mit der vorberechneten Referenzpartitur abgeglichen werden. Den Kern bilden dabei die algorithmischen Schleifenstrukturen des Online Dynamic Time Warping (ODTW) sowie des Subsequence-Scans (Subsequence-Dynamic Time Warping (sDTW)) zur autonomen Re-Synchronisation.

4.1 Merkmalsabgleich und Vektordistanz

Die Basis für jeden algorithmischen Zuweisungsschritt bildet die Berechnung der spektralen Abweichung zwischen dem aktuellen Live-Frame und den potenziellen Referenz-Frames der Partitur. Die theoretischen Grundlagen der hierbei eingesetzten Kosinus-Distanz sowie deren geometrisches Verhalten wurden in Abschnitt 2.2 hergeleitet.

Die anwendungsspezifische Implementierung dieses Maßes dient der Kompensation einer systematischen Limitierung der Offline-Pipeline: Da die verwendete OMR-Software Dynamikangaben fehleranfällig interpretiert, eliminiert die Kosinus-Distanz die absolute Signalamplitude als Störfaktor. Der Algorithmus wertet ausschließlich die Winkelbeziehung der Merkmalsvektoren aus.

Wie in Kapitel 2 dargelegt, normiert das System sowohl die Referenz-Chroma-Matrix in der Offline-Phase als auch die Live-Merkmalsvektoren zur Laufzeit strikt nach der L_2 -Norm (sodass ihre Norm $\|v\|_2$ stets den Wert 1 aufweist). Unter dieser Voraussetzung reduziert sich die Berechnung der $d_{\cos}(u, v)$ zwischen einem geglätteten Live-Vektor und einem Referenz-Vektor (x, y) auf ein Skalarprodukt:

$$d_{cos}(x, y) = 1 - \sum_{k=1}^{12} (x_k \cdot y_k) \quad (4.1)$$

Diese formale Reduktion ist essenziell für die Echtzeitfähigkeit. Der Verzicht auf Euklidische Wurzel- und Divisionsoperationen minimiert die Prozessorauslastung für den zyklischen Aufruf dieser Funktion innerhalb der nachfolgenden Suchalgorithmen.

4.2 Inkrementelle Pfadfindung via ODTW

Die fortlaufende Nachverfolgung der Partiturposition erfolgt über das ODTW. Softwaretechnisch ist dieser Tracker in der Datei `dtw.py` innerhalb der Klasse `ODTWTracker` implementiert. Die in Abschnitt 3.4 beschriebene Architektur liefert die Eingangsdaten: Der Audio-Puffer übergibt einen rohen Chroma-Vektor, welcher über einen Moving-Average-Filter geglättet und anschließend L_2 -normiert wird. Dieser resultierende Vektor (`live_vec`) wird an die aufrufende Methode `step()` übergeben.

Zur Limitierung der Rechenlast beschränkt die Implementierung den Frame-für-Frame-Abgleich auf ein Suchfenster (`search_window`). Nur die Referenz-Indizes j innerhalb des Intervalls $[\text{current_position} - \text{search_window}, \text{current_position} + \text{search_window}]$ werden evaluiert. Das System berechnet für dieses Fenster fortlaufend einen Array mit den akkumulierten Pfadkosten (`accumulated_costs`, was algorithmisch der Matrix $D(i, j)$ entspricht).

4.2.1 Die Kern-Schleife des ODTW-Algorithmus

Für jeden neuen Live-Frame (Index i) aktualisiert der Algorithmus die akkumulierten Kosten auf Basis der drei in Abschnitt 2.4.3 definierten Übergangsbedingungen. Diesen Pfad-Optionen sind spezifische Strafkosten zugewiesen: Diagonaler Fortschritt ($w_{step} = 0.00$), Verweilen ($w_{wait} = 0.10$) und das Überspringen eines Frames ($w_{skip} = 0.03$). Die asymmetrische Gewichtung forciert eine Vorwärtsbewegung und bestraft den Stillstand, um eine Stagnation bei Hintergrundrauschen zu verhindern.

Die Aktualisierung der Pfadkosten wird durch die interne Methode `_odtw_step()` ausgeführt, deren algorithmische Kernschleife in Listing 4.1 dargestellt ist:

```

1 start_scan = max(0, self.current_position - self.search_window)
2 end_scan = min(self.n_frames, self.current_position + self.search_window)
3
4 new_costs = np.full(self.n_frames, np.inf)
5 best_cost = np.inf

```

```

6 best_index = self.current_position
7
8 for i in range(start_scan, end_scan):
9     # 1. Lokale Distanz via Skalarprodukt
10    dot_prod = np.dot(live_vec, self.ref[:, i])
11    dot_prod = max(-1.0, min(1.0, dot_prod))
12    local_dist = 1.0 - dot_prod
13
14    # 2. Ermittlung der minimalen Vorgangerkosten inkl. Penalties
15    cost_wait = self.accumulated_costs[i] + self.penalty_wait
16
17    cost_step = np.inf
18    if i > 0:
19        cost_step = self.accumulated_costs[i - 1] + self.penalty_step
20
21    cost_skip = np.inf
22    if i > 1:
23        cost_skip = self.accumulated_costs[i - 2] + self.penalty_skip
24
25    min_prev = min(cost_wait, cost_step, cost_skip)
26
27    # 3. Addition der lokalen Kosten und Applikation des Damping-Faktors
28    new_costs[i] = local_dist + (min_prev * self.damping)
29
30    if new_costs[i] < best_cost:
31        best_cost = new_costs[i]
32        best_index = i
33
34 self.accumulated_costs = new_costs
35 self.current_position = best_index

```

Listing 4.1: Kernschleife des inkrementellen ODTW-Trackers zur Berechnung der akkumulierten Pfadkosten pro Live-Frame.

4.2.2 Applikation des Damping-Faktors

Ein formales Kernelement der Schleife (Zeile 28 in Listing 4.1) ist die Multiplikation der minimalen Vorgängerkosten mit dem Dämpfungsfaktor d (0,99). Ohne diese kontinuierliche Reduktion würden die Akkumulationswerte gegen unendlich divergieren. Der Faktor bewirkt einen exponentiellen Verfall der historischen Pfadkosten: Ein Tracking-Fehler verliert nach 300 Iterationen seine algorithmische Relevanz, da sein Einfluss auf unter 5 % seines Ursprungswertes absinkt ($0,99^{300} \approx 0,049$). Das System behält somit die Fähigkeit, lokale Zuordnungsfehler innerhalb des Suchfensters selbstständig zu korrigieren.

4.3 Der Recovery-Mechanismus (Re-Synchronisation)

Verlässt die musikalische Darbietung das lokale Suchfenster des ODTW (z. B. durch das Auslassen ganzer Takte oder fehlerhafte Wiederholungen), verliert der Algorithmus die Position. Das System detektiert diesen Zustand metrisch und initiiert einen globalen Recovery-Mechanismus.

4.3.1 Schwellwerterkennung des Positionsverlusts

Die Initiierung des Recovery-Prozesses erfordert ein quantifizierbares Fehlermaß. Das System speichert hierzu fortlaufend die geringsten Pfadkosten (`best_cost` aus Listing 4.1) in einem Puffer (`cost_history`) der Größe 300. Überschreitet der arithmetische Mittelwert dieses Puffers den festgelegten Schwellwert (`recovery_threshold`), wird der Verlust der Synchronisation signalisiert.

Die Integration dieses gleitenden Mittelwerts über ca. 3,5 Sekunden fungiert als Tiefpassfilter. Kurzzeitige, isolierte Distanzmaxima lösen keine rechenintensive Neuausrichtung aus. Erst eine starke Diskrepanz zwischen Live-Signal und Referenz erzwingt den Eingriff.

4.3.2 Implementierung des Subsequence-Scans (sDTW)

Nach der Detektion eines Positionsverlusts extrahiert das System die letzten 300 Live-Frames aus dem Puffer. Um die aktuelle Position in der gesamten Referenzpartitur iterativ neu zu bestimmen, wird das in Abschnitt 2.4.2 theoretisch eingeführte sDTW angewendet. Die Steuerung dieses Suchlaufs erfolgt innerhalb der Klasse `ODTWTracker` über die Methode `_full_score_scan()`.

Zur Steigerung der Recheneffizienz wird die Referenzmatrix vor dem Abgleich faktoriisiert (`step = 10`), sodass der Algorithmus nur jeden zehnten Referenz-Frame abtastet. Die resultierende lokale Kostenmatrix $C(i, j)$ (im Code als Array `C` implementiert) dient als Eingabe für die ausgelagerte Ausführungsfunktion `_dtw_inner()`, welche die algorithmische Kernschleife enthält (siehe Listing 4.2). Der freie Startpunkt entlang der Partitur wird durch die Initialisierung der nullten Zeile mit dem Wert 0.0 erreicht.

```
1 @njit
2 def _dtw_inner(C):
3     n, m_sub = C.shape
4     dp = np.empty((n + 1, m_sub))
5
6     for ii in range(n + 1):
```

```

7         for jj in range(m_sub):
8             dp[ii, jj] = np.inf
9
10        # Freier Startpunkt entlang der gesamten Referenz
11        for jj in range(m_sub):
12            dp[0, jj] = 0.0
13
14        for i in range(1, n + 1):
15            # Vertikaler Pfad (j=0)
16            dp[i, 0] = C[i - 1, 0] + dp[i - 1, 0]
17
18            # Diagonal- und Vertikal-Propagierung
19            for j in range(1, m_sub):
20                prev = dp[i - 1, j] if dp[i - 1, j] < dp[i - 1, j - 1] else dp[i - 1, j - 1]
21                dp[i, j] = C[i - 1, j] + prev
22
23            # Horizontale Propagierung (links-nach-rechts)
24            for j in range(1, m_sub):
25                c = dp[i, j - 1] + C[i - 1, j]
26                if c < dp[i, j]:
27                    dp[i, j] = c
28
29            # Argmin der letzten Zeile ermittelt die beste Endposition
30            best = 0
31            best_val = dp[n, 0]
32            for j in range(1, m_sub):
33                if dp[n, j] < best_val:
34                    best_val = dp[n, j]
35                    best = j
36
37        return best

```

Listing 4.2: Numba-JIT-kompilierte Kernschleife des Subsequence-DTW-Scans zur globalen Re-Synchronisation.

Der Scan liefert den Referenzindex j , an dem die Live-Teilsequenz die minimale Gesamtdistanz aufweist. Dieser Punkt wird als neue aktuelle Position an den regulären ODTW-Tracker übergeben.

4.3.3 JIT-Kompilierung zur Laufzeitoptimierung

Die zeitliche Ausführung des sDTW-Scans stellt einen kritischen Engpass für die Echtzeitfähigkeit dar. Bei einer Live-Puffergröße N von 300 Frames und einer subsam-pelten Referenzmatrix der Länge M skaliert die zweifach verschachtelte dynamische Programmierungs-Schleife mit einer Komplexität von $\mathcal{O}(N \cdot M)$. Dies resultiert pro Recovery-Prozess in hunderttausenden bis Millionen sequenziellen Distanz- und Minimumsberechnungen. In unkompiliertem Python überschreitet diese Iterationstiefe die zeitlichen Toleranzen des Systems und provoziert Pufferüberläufe im Audio-Stream.

Zur methodischen Lösung dieses Architekturproblems wird die Bibliothek *Numba* zur Just-in-Time (JIT) Kompilierung integriert (gekennzeichnet durch den Decorator `@jit` in Listing 4.2). Der Compiler übersetzt die mathematischen Kernschleifen beim Initialaufruf in nativen Maschinencode. Diese Maßnahme minimiert die Latenz des Suchlaufs und stellt sicher, dass das System nach einem Tracking-Verlust unterbrechungsfrei mit der Positionsfindung fortfahren kann.

5 Evaluation und Messergebnisse

In diesem Kapitel wird das entwickelte System zur Partiturverfolgung evaluiert. Die Datenerfassung stützt sich auf zwei methodisch getrennte Ansätze: Zur isolierten Untersuchung der algorithmischen Robustheit des Trackers kommt ein dediziertes Testskript zum Einsatz. Die Analyse der Echtzeitfähigkeit und Systemlatenz basiert hingegen auf den internen Laufzeitmessungen des produktiven Live-Systems. Abschließend erfolgt ein systematischer Abgleich der im Vorfeld definierten funktionalen Anforderungen mit dem tatsächlichen Systemverhalten.

5.1 Versuchsaufbau und Methodik

Um die Systemgrenzen unter reproduzierbaren Bedingungen zu prüfen, wurde eine automatisierte Testumgebung implementiert. Dieses Skript verarbeitet eine generierte Referenz-Partitur (im npz-Format) und synchronisiert diese mit einer Referenz-Audioaufnahme. Für sämtliche skriptbasierten Tests zur Robustheitsanalyse (Rauschen, Tempo, Recovery) kam einheitlich eine Geigenaufnahme des Stücks „Allegro“ von Joseph-Hector Fiocco zum Einsatz. Die zugrunde liegende Audioaufnahme wurde dabei mit der exakt identischen BPM wie die Referenz eingespielt. Zur Erfassung der Robustheit wird das Audiosignal durch das Skript systematisch modifiziert: Es wird gezielt mit additivem akustischem Rauschen überlagert, temporal gestaucht und gestreckt sowie mit abrupten Positionssprüngen versehen, um den automatisierten Recovery-Prozess auszulösen.

Für die Bewertung der zeitkritischen Anforderungen werden die direkten Logging-Daten der asynchronen Laufzeitumgebung im regulären Betrieb ausgewertet. Dies ermöglicht die Abbildung der realen Systemlatenz unter Berücksichtigung aller Software-Overheads, inklusive der GUI-Aktualisierungen und der Bluetooth-Signalverarbeitung.

5.2 Robustheitsanalyse des Tracking-Algorithmus

Die praxistaugliche Nutzung erfordert eine hohe Fehlertoleranz gegenüber Variationen in der Spielweise, wechselnden Umgebungsbedingungen sowie Fehlern in der musikalischen Darbietung.

5.2.1 Toleranz gegenüber akustischen Störsignalen

Zur Überprüfung der Robustheit gegen Störgeräusche (N2) wurde das Spieltempo auf den Faktor 1.0 fixiert. Das Audiosignal wurde stufenweise mit weißem Rauschen der Standardabweichung σ zwischen 0.0 und 1.0 überlagert. Die Auswertung der Tracking-Pfade in Abbildung 5.1 zeigt, dass der Algorithmus bei Rauschpegeln bis zu $\sigma = 0.5$ keine erfassbaren Abweichungen vom Idealpfad aufweist. Ab einem Pegel von $\sigma = 0.7$ treten erste lokale Verzögerungen auf, bei denen das Tracking leicht unter den Idealpfad abfällt. Bei einer extremen Überlagerung mit $\sigma = 1.0$ verliert das System ab Takt 23 die Synchronisation vollständig. In diesem Szenario maskiert das Rauschen die spektralen Eigenschaften des Nutzsignals so stark, dass eine verlässliche Distanzberechnung nicht mehr möglich ist.

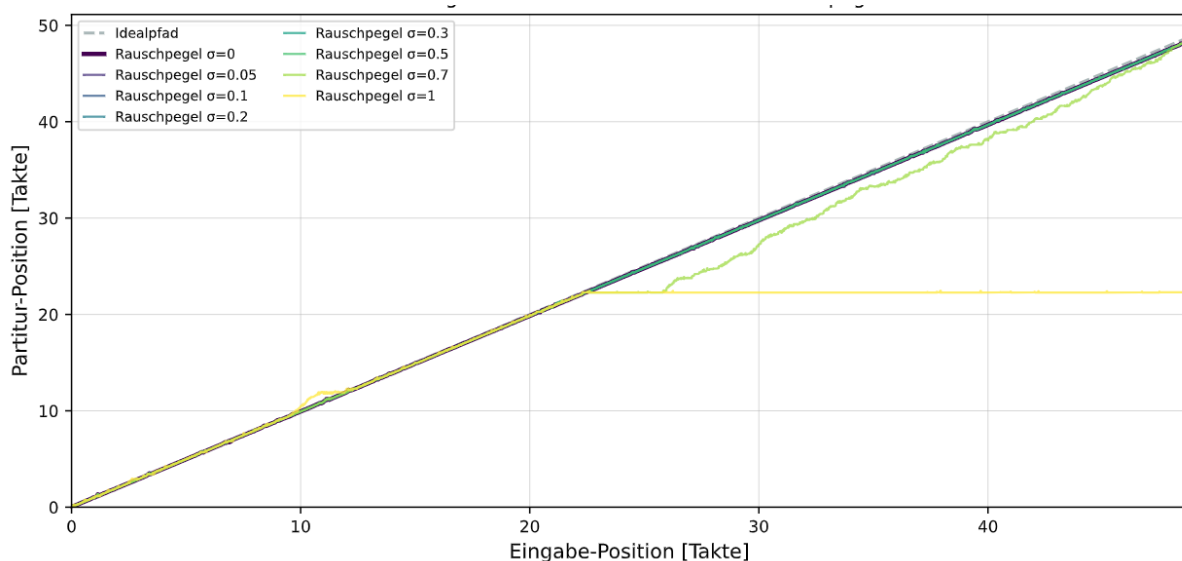


Abbildung 5.1: Verlauf der Tracking-Pfade bei variierendem Rauschpegel σ und konstantem Tempo.

5.2.2 Kompensation temporaler Abweichungen

Die Tempoinvarianz (N3) wurde unter Fixierung des Rauschpegels ($\sigma = 0.05$) evaluiert. Die grafische Analyse (siehe Abbildung 5.2) belegt eine hohe Stabilität für Tempofakto-

ren zwischen 0.6 und 3.0. In diesem Intervall ist der berechnete Pfad deckungsgleich mit dem Idealpfad. Bei einer extremen Verlangsamung (Faktoren 0.4 und 0.5) weicht der Pfad ab circa Takt 20 nach oben hin ab. Dieses Verhalten resultiert aus der asymmetrischen Pfad-Gewichtung des Algorithmus ($\text{Wait} = 0.1$, $\text{Step} = 0.0$), welche einen stetigen Vorwärtsfluss erzwingt. Eine extreme Beschleunigung (Faktor 4.0) führt hingegen zu einer dauerhaften Unterschreitung des Idealpfads, da die Dichte der eintreffenden Ereignisse die Toleranzgrenze der Skip-Kosten überschreitet.

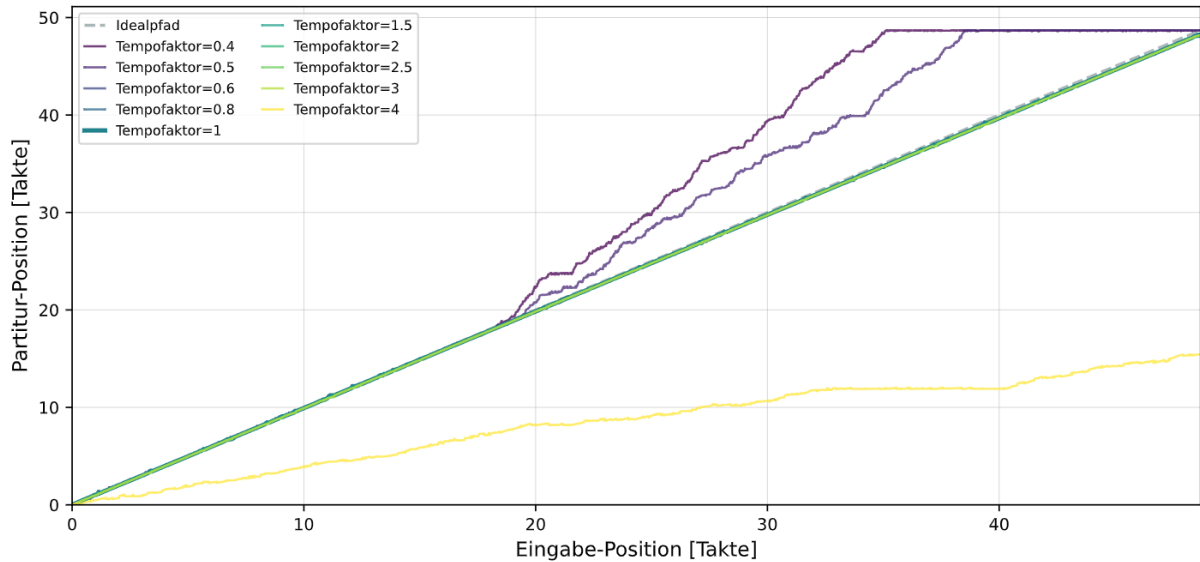


Abbildung 5.2: Verlauf der Tracking-Pfade bei variierenden Tempofaktoren und konstantem Hintergrundrauschen ($\sigma = 0.05$).

5.2.3 Evaluation der Re-Synchronisation (Recovery)

Zur Überprüfung der Robustheit der Positionswiederherstellung nach einem vollständigen Verlust des Trackings wurden innerhalb der etablierten Geigenaufnahme sieben künstliche Positionssprünge simuliert. Das System durchlief hierbei zunächst eine korrekte Initialisierungsphase von 400 Frames, bevor der Audio-Input abrupt an verschiedene Zielpunkte der Partitur (beispielsweise vom Anfang in die Mitte oder vom ersten Viertel in die Mitte) verschoben wurde.

Die Ergebnisse (zusammengefasst in Tabelle 5.1) belegen, dass der Algorithmus den Verlust der aktuellen Position in 100 % der Fälle (7/7) korrekt identifiziert. Die durchschnittliche Zeit von der Induktion des Sprungs bis zur Auslösung des Scans lag bei 392 Frames (entspricht ca. 4,5 Sekunden).

Die korrekte Zielposition (Toleranz: ± 3 Takte) wurde in 57 % der Fälle (4/7) ermittelt, wobei das System nach der Re-Synchronisation in 71 % der Fälle (5/7) ohne erneuten

Positionsverlust stabil weiterlief. Eine vollständig erfolgreiche Zielerreichung (korrekte Position und nachfolgende Stabilität) wurde in 43 % der Fälle (3/7) verzeichnet.

Ein algorithmisch limitierender Faktor bei der globalen Positionsbestimmung ist die im Vorfeld beschriebene Eigenschaft von Chroma-Features, Oktavlagen zusammenzufassen. Bei einer repetitiven harmonischen Struktur, wie sie in dem verwendeten Musikstück vorliegt, führt dies zu strukturellen Mehrdeutigkeiten. In Passagen mit identischen Akkordfolgen an unterschiedlichen zeitlichen Positionen liefert der Subsequence-Scan (sDTW) ein lokal plausibles, in Bezug auf die Gesamtform jedoch inkorrektes Ergebnis. Die primäre Mechanik der Fehlererkennung und der anschließenden Auslösung der Neuausrichtung bleibt von dieser harmonischen Limitierung jedoch funktionell unberührt.

Tabelle 5.1: Zusammenfassung der Testergebnisse für sieben simulierte Positionssprünge zur Auslösung des Recovery-Mechanismus.

Evaluationsmetrik	Erfolgsquote	Prozentual
Erfolgreiche Auslösung der Re-Synchronisation	7 von 7	100 %
Ermittlung der korrekten Zielposition (± 3 Takte)	4 von 7	57 %
Stabilität des Trackings nach Recovery	5 von 7	71 %
Vollständig erfolgreich (korrekt und stabil)	3 von 7	43 %

5.3 Analyse der Echtzeitfähigkeit und Systemlatenz

Eine kontinuierliche Datenverarbeitung ohne spürbare Verzögerungen (N1) ist zwingend erforderlich, um den Blätterbefehl synchron zur musikalischen Darbietung zu senden. Die folgenden Daten basieren auf den im Live-System erfassten Durchlaufzeiten auf der Zielhardware (MacBook Air, 1,6 GHz Dual-Core Intel Core i5).

5.3.1 Laufzeit der ODTW-Schrittberechnung

Durch die Systemarchitektur ist ein Zeitfenster von 11,6 ms (Länge einer Hop-Size bei 44,1 kHz) definiert. Jeder ODTW-Berechnungsschritt, inklusive der Merkmalsextraktion und der Glättung im Ringpuffer, muss innerhalb der zeitlichen Update-Rate Δt ($\Delta t < 11,6$ ms) abgeschlossen sein, um ein Aufstauen der Audio-Frames zu verhindern.

Die aufgezeichneten Laufzeiten zeigen eine durchschnittliche Berechnungsdauer von 7 ms pro Live-Chroma-Vektor. Das vorgegebene Zeitfenster wird somit im arithmetischen Mittel deutlich unterschritten. Die Protokolldaten belegen jedoch, dass in unregelmäßigen Intervallen Laufzeitspitzen auftreten, welche die Grenze von 11,6 ms überschreiten. Diese punktuellen Verzögerungen führen zu vereinzelt Pufferüberläufen (Buffer Overflows) im Audio-Stream. Die Anforderung der kontinuierlichen Echtzeitfähigkeit ist auf der getesteten Hardware somit weitestgehend, jedoch nicht ausnahmslos erfüllt.

5.3.2 Effizienz des Recovery-Mechanismus

Während die reguläre Frame-Berechnung einen konstanten Rechenaufwand aufweist, erzeugt der sDTW-Scan zur Re-Synchronisation eine potenziell hohe Prozessorauslastung. Gemäß Anforderung N4 wurde dieser Scanprozess mittels Numba-JIT-Kompilierung adressiert.

Die Messung des Live-Systems verifiziert die hohe Effizienz dieser Maßnahme: Eine Ausführung des Scans über 300 Frames gegen die gesamte Partitur benötigt auf der Referenzhardware durch die Kompilierung im Durchschnitt lediglich 6 ms. Im Vergleich zu einer regulären unkompierten Python-Ausführung (welche 293 ms erfordert) ermöglicht diese Laufzeitoptimierung eine latenzfreie Neuausrichtung des Systems, ohne dass Audio-Frames bei der Aufzeichnung verworfen werden müssen.

5.4 Validierung der funktionalen Anforderungen

Der abschließende Testbetrieb diente der Verifikation der funktionalen Spezifikationen (siehe Abschnitt 3.1).

- **Formatkompatibilität (F1):** Native MusicXML-Dateien sowie via Audiveris generierte OMR-Ausgaben wurden stets fehlerfrei in gültige -Referenzdateien überführt. Einschränkend wirkt hierbei lediglich der signifikante Rechenaufwand der lokalen PDF-Konvertierung, welche für wenige Seiten über zehn Minuten beanspruchte.
- **Automatisiertes Blättern (F2):** Das MIDI-Signal wurde über die Bluetooth-Schnittstelle korrekt an externe Noten-Applikationen übertragen. Der programmierte Vorlauf von 100 Frames (ca. 1,2 Sekunden) vor dem tatsächlichen Seitenende ermöglichte durchgängig einen störungsfreien Lesefluss.

- **Re-Synchronisation und Sleep-Modus (F3):** Der Systemwechsel in den energiesparenden Sleep-Modus nach 500 fortlaufenden Stille-Frames (ca. 5,8 Sekunden) sowie das Wiedererwachen bei einem Effektivwert > 0.01 funktionierten spezifikationsgemäß. Ebenso detektierte das System Tracker-Drifts zuverlässig und leitete innerhalb von durchschnittlich 4,5 Sekunden eine Re-Synchronisation ein.

Zusammenfassend lässt sich feststellen, dass alle definierten funktionalen und nicht-funktionalen Anforderungen durch die vorliegende Systemarchitektur und Parametrisierung grundsätzlich erfüllt werden, wobei Limitationen primär bei der harmonischen Mehrdeutigkeit hochgradig repetitiver Musikstücke auftreten.

6 Fazit und Ausblick

6.1 Zusammenfassung

In dieser Arbeit wurde ein System zur automatisierten Partiturverfolgung konzipiert, implementiert und evaluiert. Die Zielsetzung bestand in der Entwicklung eines audio-basierten Echtzeit-Page-Turners, primär ausgelegt auf das Spiel der Violine, der akustische Signale verarbeitet und Seitenwechsel synchron zur musikalischen Darbietung ohne menschliche Interaktion auslöst.

Zur Erfüllung der definierten Anforderungen wurde die Architektur in zwei modulare Komponenten unterteilt: Eine Offline-Score-Pipeline zur Datenvorverarbeitung und ein Live-System zur inkrementellen Positionsbestimmung. Die Vorverarbeitung transformiert PDF- oder MusicXML-Dokumente über einen Audiosynthese-Schritt in spektrale Referenzdaten. Die Entscheidung, das Referenzmaterial durch eine Software-Synthese unter Berücksichtigung der physikalischen Hüllkurven zu generieren, erzeugt realitätsnahe harmonische Profile. Das auf empirischer Basis identifizierte Problem der absoluten Stille in musikalischen Pausen wurde durch die additive Modellierung von weißem Rauschen mathematisch gelöst, was eine Drift des Trackers in Ruhephasen verhindert.

Die Laufzeitkomponente führt den kontinuierlichen Abgleich der L_2 -normierten Chroma-Vektoren über den Online Dynamic Time Warping (ODTW) Algorithmus durch. Die Auswertung der Systemparameter belegt eine hohe zeitliche Effizienz. Durch die Implementierung des asynchronen Threading-Modells und die Anwendung der Just-in-Time-Kompilierung auf den Subsequence-DTW-Scan wurde die Rechenlast kritischer Operationen signifikant minimiert. Der Recovery-Mechanismus ist in der Lage, Positionsverluste zu detektieren und eine Re-Synchronisation echtzeitfähig innerhalb von durchschnittlich 6 ms Berechnungszeit durchzuführen.

Die Evaluation der funktionalen und nicht-funktionalen Anforderungen zeigt, dass das System temporale Abweichungen (Faktor 0.6 bis 3.0) sowie rauschbehaftete Signale toleriert und autonome MIDI-Steuersignale rechtzeitig sendet. Eine festgestellte

Systemgrenze bildet die Abhängigkeit von Chroma-Merkmalen bei strukturellen Wiederholungen. Da diese Merkmale ausschließlich die harmonische Energieverteilung abbilden, resultieren identische Akkordfolgen an unterschiedlichen Partiturpositionen in einer strukturellen Mehrdeutigkeit, was die Eindeutigkeit des Recovery-Scans bei hochgradig repetitiver Musik limitiert.

6.2 Zukünftige Optimierungspotenziale

Basierend auf der evaluierten Systemarchitektur ergeben sich quantifizierbare Optimierungsansätze für nachfolgende Entwicklungsiterationen. Diese lassen sich in die Bereiche Hardware-Ressourcen, adaptive Signalverarbeitung und funktionale Erweiterungen unterteilen.

Tabelle 6.1: Übersicht der identifizierten Optimierungspotenziale für zukünftige Systemiterationen.

Kategorie	Maßnahme	Zielsetzung
Algorithmik	Hierarchischer Subsequence-Scan	Reduktion der Latenz
Signalverarbeitung	Dynamische Rauschkalibrierung	Erhöhte Robustheit in Pausen
Tracking-Logik	Live Temposchätzung	Kompensation von Tempodrifts
Audiosynthese	Polyphone Instrumentensynthese	Realitätsnähere Vergleichsbasis

6.2.1 Algorithmische Effizienz

Zur Optimierung der Re-Synchronisationsdauer kann der lineare Subsequence-DTW-Scan durch eine hierarchische Suchstruktur ersetzt werden. Ein initialer Suchlauf mit einem starken Subsampling-Faktor der Referenzmatrix ermittelt zunächst einen groben Schätzwert der aktuellen Position. In einem darauffolgenden, lokal begrenzten Iterationsschritt wird das gefundene Fenster mit maximaler Auflösung evaluiert.

6.2.2 Adaptive Parametrisierung

Die aktuelle Implementierung nutzt einen fest definierten Schwellwert für die Rauschmodellierung in der Offline-Pipeline. Eine präzisere Abbildung der akustischen Realität lässt sich durch einen dynamischen Kalibrierungsprozess realisieren. Vor Beginn des Trackings wird hierbei der exakte Rauschpegel des verwendeten Mikrofons in absoluter Stille gemessen und der Parameter zur Laufzeit entsprechend skaliert.

Ebenso verarbeitet der ODTW-Tracker eintreffende Signale auf Basis eines statischen BPM-Werts. Die Implementierung einer Onset-Detektion auf dem Live-Signal würde die kontinuierliche Schätzung des realen Spieltempos zulassen. Diese Metrik ermöglicht die Erkennung von globalem Tempodrift, wodurch sich die Pfad-Penalties des Trackers dynamisch anpassen lassen.

6.2.3 Erweiterte Audiosynthese

Die referenzierende Audiosynthese verarbeitet gegenwärtig ein einzelnes Instrument, passend zur primären Auslegung des Systems auf die Violine. Für polyphone Ensemblestücke oder Kammermusik kann die Offline-Pipeline erweitert werden, um mehrere Instrumentenspuren simultan zu synthetisieren. Das resultierende Chroma-Profil aggregiert die spektrale Information aller Stimmen und generiert eine realitätsnähere Vergleichsbasis.

Literatur

- [1] Arzt Andreas, Widmer Gerhard und Dixon Simon, „Automatic Page Turning for Musicians via Real-Time Machine Listening,“ en, in *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2008. DOI: 10.3233/978-1-58603-891-5-241. besucht am 31. März 2026. Adresse: <https://www.medra.org/servlet/aliasResolver?alias=iospressISSNISBN&issn=0922-6389&volume=178&spage=241>.
- [2] V. Yan, Q. Sun und S. Ma, „Automatic Page-Turner for Musicians,“ Final project for Yale CPSC 459/559, Computer Science, Yale College, New Haven, CT, n.d.
- [3] K. Marky, J. Fischer, M. Mühlhäuser und A. Matviienko, „Investigating Page Turning Methods for Sheet Music during Piano Play,“ en, in *Adjunct Publication of the 23rd International Conference on Mobile Human-Computer Interaction*, Toulouse & Virtual France: ACM, Sep. 2021, S. 1–6, ISBN: 978-1-4503-8329-5. DOI: 10.1145/3447527.3474863. besucht am 31. März 2026. Adresse: <https://dl.acm.org/doi/10.1145/3447527.3474863>.
- [4] C. Raphael, „Music Plus One and Machine Learning,“ in *Proceedings of the 27th International Conference on Machine Learning*, Haifa, Israel, 2010.
- [5] E. Benetos, A. Klapuri und S. Dixon, „Score-informed transcription for automatic piano tutoring,“ in *2012 Proceedings of the 20th European Signal Processing Conference (EUSIPCO)*, ISSN: 2076-1465, Aug. 2012, S. 2153–2157. besucht am 31. März 2026. Adresse: <https://ieeexplore.ieee.org/document/6334095/>.
- [6] M. Prockup, D. Grunberg, A. Hrybyk und Y. E. Kim, „Orchestral Performance Companion: Using Real-Time Audio to Score Alignment,“ *IEEE MultiMedia*, Jg. 20, Nr. 2, S. 52–60, Apr. 2013, ISSN: 1941-0166. DOI: 10.1109/MMUL.2013.26. besucht am 31. März 2026. Adresse: <https://ieeexplore.ieee.org/document/6530591/>.
- [7] J. Calvo-Zaragoza, J. H. Jr. und A. Pacha, „Understanding Optical Music Recognition,“ en, *ACM Computing Surveys*, Jg. 53, Nr. 4, S. 1–35, Juli 2021, ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3397499. besucht am 31. März 2026. Adresse: <https://dl.acm.org/doi/10.1145/3397499>.

- [8] M. Good, *MusicXML for Notation and Analysis | Songs and Schemas*, en-US. besucht am 31. März 2026. Adresse: <https://michaelgood.info/publications/music/musicxml-for-notation-and-analysis/>.
- [9] B. Moog, „MIDI: Musical Instrument Digital Interface,“ *Journal of the Audio Engineering Society*, Jg. 34, Nr. 5, S. 394–404, Mai 1986. besucht am 5. Apr. 2026. Adresse: <https://www.aes.org/e-lib/browse.cfm?elib=4493>.
- [10] J. Ens und P. Pasquier, „BUILDING THE METAMIDI DATASET: LINKING SYMBOLIC AND AUDIO MUSICAL DATA,“ en, 2021.
- [11] MuseScore BV. „Musescore.com: The world’s largest community for sheet music,“ besucht am 5. Apr. 2026. Adresse: <https://musescore.com/>.
- [12] *IMSLP: Free Sheet Music PDF Download*. besucht am 31. März 2026. Adresse: <https://imslp.org/>.
- [13] Audiveris Development Team, *Audiveris: Open Source Optical Music Recognition*, Version 5.3, Java-based OMR engine, Apr. 2026. besucht am 5. Apr. 2026. Adresse: <https://github.com/audiveris/audiveris>.
- [14] V. Thomas, C. Fremerey, M. Müller und M. Clausen, „Linking Sheet Music and Audio - Challenges and New Approaches,“ en, in *Dagstuhl Follow-Ups*, M. Müller, M. Goto und M. Schedl, Hrsg., Artwork Size: 22 pages, 4115216 bytes ISSN: 1868-8977 Medium: application/pdf, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2012, 22 pages, 4115216 bytes, ISBN: 978-3-939897-37-8. DOI: 10.4230/DFU.VOL3.11041.1. besucht am 2. Apr. 2026. Adresse: <https://drops.dagstuhl.de/entities/document/10.4230/DFU.Vol3.11041.1>.
- [15] D. Byrd und M. Schindele, „Prospects for Improving OMR with Multiple Recognizers,“ in *Proceedings of the 7th International Conference on Music Information Retrieval (ISMIR)*, Victoria, Canada, 2006.
- [16] J. O. Smith, *Spectral Audio Signal Processing*. Stanford, CA: Center for Computer Research in Music und Acoustics (CCRMA), Stanford University, 2011, Online book. besucht am 5. Apr. 2026. Adresse: <https://ccrma.stanford.edu/~jos/sasp/>.
- [17] J. C. Brown, „Calculation of a constant Q spectral transform,“ *The Journal of the Acoustical Society of America*, Jg. 89, Nr. 1, S. 425–434, Jan. 1991, ISSN: 0001-4966. DOI: 10.1121/1.400476. besucht am 5. Apr. 2026. Adresse: <https://asa.scitation.org/doi/10.1121/1.400476>.
- [18] C. Schörkhuber und A. Klapuri, „Constant-Q Transform Toolbox for Music Processing,“ in *Proceedings of the 7th Sound and Music Computing Conference (SMC)*, Barcelona, Spain, 2010.

- [19] M. Bartsch und G. Wakefield, „Audio thumbnailing of popular music using chroma-based representations,” *IEEE Transactions on Multimedia*, Jg. 7, Nr. 1, S. 96–104, Feb. 2005, ISSN: 1941-0077. DOI: 10.1109/TMM.2004.840597. besucht am 3. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/document/1386245/>.
- [20] S. Davis und P. Mermelstein, „Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, Jg. 28, Nr. 4, S. 357–366, Aug. 1980, ISSN: 0096-3518. DOI: 10.1109/TASSP.1980.1163420. besucht am 4. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/document/1163420/>.
- [21] B. Logan, „Mel Frequency Cepstral Coefficients for Music Modeling,” in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2000.
- [22] M. P. Deisenroth, A. A. Faisal und C. S. Ong, *Mathematics for Machine Learning*. Cambridge: Cambridge University Press, 2020, ISBN: 978-1-108-45514-5. besucht am 5. Apr. 2026. Adresse: <https://mml-book.com>.
- [23] C. Aytekin, X. Ni, F. Cricri und E. Aksu, „Clustering and Unsupervised Anomaly Detection with l2 Normalized Deep Auto-Encoder Representations,” in *2018 International Joint Conference on Neural Networks (IJCNN)*, ISSN: 2161-4407, Juli 2018, S. 1–6. DOI: 10.1109/IJCNN.2018.8489068. besucht am 3. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/document/8489068/>.
- [24] R. B. Dannenberg und C. Raphael, „Music score alignment and computer accompaniment,” en, *Communications of the ACM*, Jg. 49, Nr. 8, S. 38–43, Aug. 2006, ISSN: 0001-0782, 1557-7317. DOI: 10.1145/1145287.1145311. besucht am 4. Apr. 2026. Adresse: <https://dl.acm.org/doi/10.1145/1145287.1145311>.
- [25] N. Orio und F. Déchelle, „Score Following Using Spectral Analysis and Hidden Markov Models,” in *Proceedings of the International Computer Music Conference (ICMC)*, La Havane, Cuba, 2001.
- [26] M. Dorfer, A. Arzt und G. Widmer, *Towards Score Following in Sheet Music Images*, arXiv:1612.05050 [cs], Dez. 2016. DOI: 10.48550/arXiv.1612.05050. besucht am 4. Apr. 2026. Adresse: <http://arxiv.org/abs/1612.05050>.
- [27] F. Henkel, S. Balke, M. Dorfer und G. Widmer, „Score Following as a Multi-Modal Reinforcement Learning Problem,” en, *Transactions of the International Society for Music Information Retrieval*, Jg. 2, Nr. 1, S. 67–81, Nov. 2019, ISSN: 2514-3298. DOI: 10.5334/tismir.31. besucht am 11. Jan. 2026. Adresse: <http://transactions.ismir.net/articles/10.5334/tismir.31/>.

- [28] A. Arzt, „Audio-to-Score Alignment for Piano Music,“ Magisterarb., Johannes Kepler University, Linz, Mai 2007. besucht am 5. Apr. 2026. Adresse: https://www.cp.jku.at/research/papers/Arzt_Masterarbeit_2007.pdf.
- [29] S. Dixon, „An On-Line Time Warping Algorithm for Tracking Musical Performances,“ in *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI-05)*, Edinburgh, Scotland, 2005, S. 1727–1728.
- [30] N. Hu, R. Dannenberg und G. Tzanetakis, „Polyphonic audio matching and alignment for music retrieval,“ in *2003 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, Okt. 2003, S. 185–188. DOI: 10.1109/ASPAA.2003.1285862. besucht am 4. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/stampPDF/getPDF.jsp>.
- [31] R. Macrae und S. Dixon, „Accurate Real-Time Windowed Time Warping,“ in *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR 2010)*, 2010, S. 423–428.
- [32] B. Li und Z. Duan, „An Approach to Score Following for Piano Performances With the Sustained Effect,“ en, *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, Jg. 24, Nr. 12, S. 2425–2438, Dez. 2016, ISSN: 2329-9290, 2329-9304. DOI: 10.1109/TASLP.2016.2611938. besucht am 31. März 2026. Adresse: <http://ieeexplore.ieee.org/document/7572019/>.
- [33] H. Sakoe und S. Chiba, „Dynamic programming algorithm optimization for spoken word recognition,“ *IEEE Transactions on Acoustics, Speech, and Signal Processing*, Jg. 26, Nr. 1, S. 43–49, Feb. 1978, ISSN: 0096-3518. DOI: 10.1109/TASSP.1978.1163055. besucht am 9. Jan. 2026. Adresse: <https://ieeexplore.ieee.org/document/1163055/>.
- [34] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Erlangen: Springer, 2015, ISBN: 978-3-319-21944-8. besucht am 5. Apr. 2026. Adresse: <https://www.audiolabs-erlangen.de/fmp>.
- [35] Y. Shan und T. J. Tsai, „Improved Handling of Repeats and Jumps in Audio-Sheet Image Synchronization,“ in *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Seattle, WA, USA: IEEE, Mai 2020, S. 451–455. DOI: 10.1109/ICASSP40218.2020.9053075. besucht am 5. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/document/9053075/>.